THE COMPLETE LARAVEL COURSE

From Absolute Beginner to Advanced (React + Laravel + Oracle/MySQL Edition)

> Written for a developer who already knows JavaScript, React, Node.js, Express, and MongoDB basics — and is now learning PHP + Laravel. Every hard word is explained in simple English the first time it appears.

TABLE OF CONTENTS

1. PHP for Laravel
2. What is Laravel?
3. Installation & Setup
4. Laravel Project Structure
5. Routing
6. Controllers
7. Blade
8. Database Fundamentals
9. Migrations
10. Models & Eloquent ORM
11. Eloquent Queries
12. Database Relationships
13. CRUD API (Products)
14. Form Request Validation
15. API Resources
16. API Pagination
17. Search, Filter & Sort
18. Authentication (Sanctum)
19. Authorization (Gates, Policies, Roles)
20. File & Image Upload
21. Laravel + React
22. Oracle + Laravel
23. Middleware
24. Service Classes
25. Repository Pattern
26. Events & Listeners
27. Jobs & Queues
28. Mail

29. Notifications
30. Error Handling
31. Security
32. Database Seeders & Factories
33. Testing
34. Artisan Command Cheat Sheet
35. Professional Project Structure
36. Git & GitHub
37. Deployment
38. Final Project — E-Commerce Management System
39. API Documentation
40. Interview Preparation (90 Questions)
41. Common Errors & Troubleshooting
42. Laravel Cheat Sheet
43. 30-Day Learning Roadmap

PART 1 — PHP FOR LARAVEL

You don't need to become a PHP expert. You only need the PHP that Laravel is built on. Every topic below connects directly to something you'll use in Laravel.

1.1 What is PHP?

Simple explanation: PHP is a programming language that runs on a server (not in the browser, like JavaScript in React). When someone visits a website built with PHP, the server runs PHP code and sends back plain HTML or JSON — the visitor never sees the PHP itself.

Laravel connection: Laravel is a PHP framework. A framework is a set of ready-made tools and rules so you don't build everything from zero — similar to how Express.js gives you routing and middleware on top of raw Node.js.

1.2 PHP Installation

You need PHP installed to run Laravel. Laravel itself will guide you to check your PHP version:

```bash
php -v
```

Explanation: This prints the installed PHP version. Laravel 11/12 requires PHP 8.2 or higher.

1.3 PHP Syntax Basics

Every PHP file (outside of pure HTML) starts with `<?php`.

```php
<?php

echo "Hello World";
```

Explanation:

- `<?php` tells the server "PHP code starts here."
- `echo` prints text (similar to `console.log`, but `echo` sends output to the response, not a terminal log).
- Every statement ends with `;` (same as JavaScript, but required, not optional).

Laravel connection: You will rarely write raw `<?php ... ?>` files yourself in API projects — Laravel handles this — but Blade templates and config files use this syntax.

1.4 Variables

```php
<?php
$name = "Ali";
$age = 22;
```

Simple explanation: A variable stores a value. In PHP, every variable name starts with `$` (unlike JavaScript's `let`/`const`).

JavaScript	PHP
`let name = "Ali";`	`$name = "Ali";`
`const age = 22;`	`$age = 22;` (PHP has no true `const` for variables, but `define()`/`const` exist for constants)

1.5 Data Types

```php
```

```php
$string = "Hello";       // text
$int = 10;                // whole number
$float = 10.5;            // decimal number
$bool = true;             // true/false
$array = [1, 2, 3];       // list
$null = null;             // empty value
```

Laravel connection: Eloquent models automatically cast database columns to these PHP types (e.g. a database `int` becomes a PHP `int`).

1.6 Strings

```php
$first = "John";
$last = "Doe";

// Concatenation (joining strings) uses a dot, not +
$full = $first . " " . $last;

// String interpolation (like JS template literals `${}`)
$greeting = "Hello, {$first}!";
```

Explanation: PHP uses `.` to join strings (JavaScript uses `+`). Double-quoted strings support `{$variable}` interpolation, similar to JS backticks.

1.7 Numbers

```php
$a = 10;
$b = 3;

echo $a + $b;   // 13
echo $a % $b;   // 1 (remainder)
echo $a ** $b;  // 1000 (power)
```

1.8 Arrays

```php
```

```php
$fruits = ["apple", "banana", "mango"];

echo $fruits[0];        // apple
$fruits[] = "grape";    // add an item (like push())
count($fruits);         // 4
```

Simple explanation: An array is an ordered list, like a JavaScript array.

1.9 Associative Arrays

```php
$user = [
    "name" => "Ali",
    "age" => 22,
];

echo $user["name"]; // Ali
```

Simple explanation: This is PHP's version of a JavaScript object `{ name: "Ali", age: 22 }`. The `=>` is like JavaScript's `:`.

Laravel connection: Laravel uses associative arrays everywhere — configuration files, validation rules, and JSON responses are built from them.

1.10 Conditions

```php
$age = 20;

if ($age >= 18) {
    echo "Adult";
} elseif ($age >= 13) {
    echo "Teen";
} else {
    echo "Child";
}
```

Same logic as JavaScript's `if/else if/else`, just `elseif` is one word.

1.11 Loops

```php
```

```php
// for loop
for ($i = 0; $i < 5; $i++) {
    echo $i;
}

// foreach loop (most common in Laravel)
$fruits = ["apple", "banana"];
foreach ($fruits as $fruit) {
    echo $fruit;
}

// foreach with associative array
foreach ($user as $key => $value) {
    echo "$key: $value";
}

// while loop
$i = 0;
while ($i < 5) {
    echo $i;
    $i++;
}
```

Laravel connection: `foreach` is used constantly in Blade views to loop over database results.

1.12 Functions

```php
function greet($name) {
    return "Hello, $name!";
}

echo greet("Ali"); // Hello, Ali!
```

1.13 Parameters

```php
```

```php
function add($a, $b = 10) {  // $b has a default value
    return $a + $b;
}

add(5);      // 15
add(5, 20);  // 25
```

1.14 Return Values

Simple explanation: `return` sends a value back from the function, the same as JavaScript. A function without `return` gives back `null`.

1.15 Include / Require

```php
require 'config.php';
include 'header.php';
```

Simple explanation: These pull the content of one PHP file into another. `require` stops the program with an error if the file is missing; `include` just gives a warning and continues.

Laravel connection: You almost never use these directly in Laravel — Laravel uses **autoloading** (via Composer) instead, so classes are loaded automatically when needed.

1.16 Classes

```php
class Car {
    // properties and methods go here
}
```

Simple explanation: A class is a blueprint for creating objects — same concept as a JavaScript `class`.

1.17 Objects

```php
$car = new Car();
```

Simple explanation: An object is a real instance created from a class, using `new`.

1.18 Constructors

```php
class Car {
    public string $brand;

    public function __construct(string $brand) {
        $this->brand = $brand;
    }
}

$car = new Car("Toyota");
echo $car->brand; // Toyota
```

Simple explanation: `__construct()` runs automatically when an object is created — same as a JS class `constructor()`. `$this` refers to the current object (like JS `this`), and PHP uses `->` instead of `.` to access properties/methods.

1.19 Properties

Properties are variables that belong to a class (like JS class fields).

```php
class Car {
    public string $brand;
    protected int $year;
    private string $vin;
}
```

Simple explanation of visibility:

- `public` — accessible from anywhere
- `protected` — accessible from this class and child classes only
- `private` — accessible only from this exact class

1.20 Methods

```php
```

```php
class Car {
    public function start() {
        return "Engine started";
    }
}
```

Simple explanation: A method is a function that belongs to a class.

1.21 Inheritance

```php
class Vehicle {
    public function move() {
        return "Moving...";
    }
}

class Car extends Vehicle {
    // Car automatically gets move()
}
```

Laravel connection: Every Eloquent Model you create `extends Model`. Every Controller `extends Controller`. This is how Laravel gives your classes built-in superpowers.

1.22 Interfaces

```php
interface PaymentGateway {
    public function pay(float $amount);
}

class StripeGateway implements PaymentGateway {
    public function pay(float $amount) {
        // real Stripe logic
    }
}
```

Simple explanation: An interface is a contract — it says "any class that implements me MUST have these methods," but doesn't provide the code itself.

Laravel connection: Used heavily in the Repository Pattern (Part 25) and dependency injection.

1.23 Traits

```php
trait Loggable {
    public function log($message) {
        echo "LOG: $message";
    }
}

class OrderService {
    use Loggable;
}
```

Simple explanation: A trait is reusable code you can "mix in" to multiple classes. PHP doesn't support multiple inheritance (a class can't `extend` two classes), so traits solve that.

Laravel connection: Laravel itself uses traits everywhere, e.g. `use HasFactory, Notifiable;` inside the `User` model.

1.24 Namespaces

```php
namespace App\Models;

class Product {
    // ...
}
```

Simple explanation: A namespace is like a folder path for your class name, so two classes can share the same name without conflict (similar to how ES modules avoid name collisions).

Laravel connection: Every single class in Laravel starts with a `namespace` matching its folder — `App\Models\Product` lives at `app/Models/Product.php`.

1.25 Exceptions

```php
```

```php
try {
    $result = 10 / 0;
} catch (\DivisionByZeroError $e) {
    echo "Error: " . $e->getMessage();
}
```

Simple explanation: Same as JS `try/catch` — code that might fail goes in `try`, and if it throws an error, `catch` handles it gracefully instead of crashing.

Laravel connection: Laravel wraps your entire application in exception handling automatically and turns errors into clean JSON responses (Part 30).

1.26 Basic OOP Concepts Summary

Concept	Simple Meaning
Class	Blueprint
Object	Real instance of a class
Property	Data stored in an object
Method	Function inside a class
Inheritance	A class reusing another class's code
Interface	A contract of required methods
Trait	Reusable shared code
Namespace	Organized "address" for a class

You now know enough PHP to learn Laravel properly.

PART 2 — WHAT IS LARAVEL?

2.1 What is Laravel?

Simple explanation: Laravel is a PHP framework for building web applications and APIs. Think of it as "Express.js, but with way more built-in features" — routing, database ORM, authentication, validation, mail, queues, and testing all come pre-integrated.

2.2 Why Laravel?

- Huge ecosystem and community
- Clean, readable syntax
- Built-in solutions for almost everything (auth, jobs, mail, caching)

- Excellent documentation
- Used heavily for freelance and enterprise backend work

2.3 Why Companies Use Laravel

- Faster development (less boilerplate than raw PHP)
- Easier to hire developers for (huge community)
- Strong security defaults (CSRF, hashing, SQL-injection protection built in)
- Great for both traditional websites and REST APIs

2.4 Laravel Advantages

- Eloquent ORM (very easy database work)
- Artisan CLI (code generation, migrations)
- Built-in authentication (Sanctum, Breeze, Jetstream)
- Queues, events, jobs, notifications built in
- Testing tools included

2.5 Laravel Disadvantages

- Can feel "magical" at first (things happen automatically, which is confusing until you understand conventions)
- Slightly heavier than a minimal framework
- PHP itself is less popular than JS/Python for new grads, so fewer learning resources than JS ecosystems

2.6 Laravel vs Express.js

Express.js	Laravel
Minimal, you add everything (auth, ORM) yourself	Batteries-included
Routing only, by default	Routing + ORM + Auth + Mail + Queues built in
`npm install` for most features	Most features ship with the framework
JavaScript/Node.js	PHP

Simple analogy: Express is like a bare frame of a car — you bolt on everything. Laravel is like a fully equipped car straight from the factory.

2.7 Laravel vs Node.js

Not a fair 1:1 comparison — Node.js is a runtime, Laravel is a framework. The real comparison is Laravel (PHP) vs Express/Nest (Node.js) as backend solutions. Laravel tends to have more built-in conventions; Node.js ecosystems tend to be more modular and JavaScript-unified across front and back end.

2.8 Laravel vs Django

Django (Python)	Laravel (PHP)
"Batteries included" like Laravel	"Batteries included"
Django ORM	Eloquent ORM
Django REST Framework for APIs	Laravel API Resources + Sanctum
Python	PHP

Both are similar in philosophy — full-featured frameworks with strong conventions.

2.9 MVC (Model-View-Controller)

Simple explanation:

- **Model** = talks to the database (e.g. `Product` model)
- **View** = what the user sees (Blade template, or in your case — a JSON response consumed by React)
- **Controller** = the traffic cop that receives the request, asks the Model for data, and returns a View/response

2.10 Request Lifecycle

text

```
1. Request enters through public/index.php
2. Laravel bootstraps the application (loads config, service providers)
3. Request goes through the HTTP Kernel (global middleware)
4. Router matches the URL to a route
5. Route-specific middleware runs (e.g. auth check)
6. Controller method executes
7. Controller talks to Model/Service
8. Response is built (JSON, HTML)
9. Response goes back through middleware
10. Response is sent to the browser/React app
```

2.11 Laravel Architecture (Your Use Case)

text

```
React (Frontend)
   ↓  HTTP Request (Axios/Fetch)
Laravel Route          (routes/api.php)
   ↓
Middleware             (checks auth, throttling, etc.)
   ↓
Controller             (receives request, calls services)
   ↓
Service                (business logic)
   ↓
Model (Eloquent)       (talks to database)
   ↓
Database               (MySQL / Oracle)
   ↓
JSON Response          (built by Controller/API Resource)
   ↓
React                  (renders the data)
```

Explanation of each layer, simply:

- **Middleware** = a layer that checks a request before it reaches the controller (e.g. "is this user logged in?").
- **Controller** = receives the request and decides what to do — it should stay thin (little logic).
- **Service** = a plain PHP class that holds business logic, keeping controllers clean.
- **Model** = represents one database table and lets you query it using PHP instead of raw SQL.

2.12 Laravel Ecosystem

- **Sanctum** — lightweight API token authentication
- **Breeze/Jetstream** — pre-built authentication scaffolding for Blade/Vue/React starter kits
- **Horizon** — dashboard for monitoring queues
- **Nova** — official admin panel (paid)
- **Forge/Vapor** — official deployment platforms
- **Pest/PHPUnit** — testing frameworks
- **Telescope** — debugging/insight tool for local development

---

PART 3 — INSTALLATION & SETUP

3.1 What You Need

Tool	Purpose
PHP (8.2+)	Runs Laravel
Composer	PHP's package manager (like `npm` for PHP)
Laravel Installer	Quickly scaffolds new projects
VS Code	Code editor
XAMPP (optional)	Bundles PHP, MySQL, Apache for local development

3.2 Installing PHP

Download from php.net, or (recommended for beginners) install **XAMPP**, which bundles PHP + MySQL + a local server together in one installer. After installing, verify:

```bash
php -v
```

3.3 Installing Composer

Simple explanation: Composer is PHP's version of `npm`. It downloads and manages your project's PHP libraries, listed in `composer.json` (like `package.json`).

Download from getcomposer.org, then verify:

```bash
composer -v
```

3.4 Installing the Laravel Installer

```bash
composer global require laravel/installer
```

Explanation: This installs a global command called `laravel`, letting you create new projects with `laravel new`.

3.5 Creating a Laravel Project

```bash
composer create-project laravel/laravel my-app
```

Explanation, line by line:

- `composer create-project` — tells Composer to scaffold a brand-new project.
- `laravel/laravel` — the official skeleton package to copy from.
- `my-app` — the folder name for your new project.

Alternative (if you installed the Laravel installer):

```bash
laravel new my-app
```

3.6 Running the Project

```bash
cd my-app
php artisan serve
```

Explanation: `artisan` is Laravel's built-in command-line tool (similar to `npm run` scripts, but built into the framework). `serve` starts a local development server, usually at `http://127.0.0.1:8000`.

3.7 The `.env` File

Simple explanation: `.env` stores environment-specific secrets and settings (database credentials, app URL, mail settings) — it should **never** be committed to Git.

```env
```

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:xxxxxxxxxxxxxxxxxxxx
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=my_app
DB_USERNAME=root
DB_PASSWORD=
```

3.8 `APP_KEY`

Simple explanation: `APP_KEY` is a secret key Laravel uses to encrypt data (sessions, cookies). It's generated automatically:

```bash
php artisan key:generate
```

3.9 Configuration

Laravel's `config/` folder holds PHP files (not `.env` directly) that read values from `.env` using `env()`. You typically edit `.env`, not the config files themselves, for day-to-day settings.

3.10 VS Code Setup

Recommended extensions:

- PHP Intelephense (autocomplete, error checking)
- Laravel Blade Snippets
- Laravel Extra Intellisense

3.11 Local Development Server Options

Option	Description
`php artisan serve`	Quick built-in dev server, no setup needed
XAMPP/Apache	Serves the app like a real production server, good for Blade-heavy apps
Laravel Sail	Docker-based environment (Laravel's official Docker setup)

PART 4 — LARAVEL PROJECT STRUCTURE

```
text

app/            → Your application code (Models, Controllers, Services...)
bootstrap/      → Framework bootstrap files (rarely touched)
config/         → All configuration files (database.php, mail.php, etc.)
database/       → Migrations, seeders, factories
public/         → The only folder exposed to the web (index.php, assets)
resources/      → Blade views, raw CSS/JS before compiling
routes/         → All route definitions (web.php, api.php)
storage/        → Logs, cached files, uploaded files
tests/          → Automated tests
vendor/         → Installed Composer packages (like node_modules)
.env            → Environment variables/secrets
artisan         → The CLI tool entry file
composer.json   → Project dependencies (like package.json)
```

`app/` — Deep Dive

```
text

app/
 ├── Http/
 │    ├── Controllers/     → handle requests
 │    ├── Middleware/      → request filters
 │    └── Requests/        → form validation classes
 ├── Models/                → Eloquent models
 ├── Providers/              → service providers (framework bootstrapping)
```

Why it exists / when you use it: This is where 90% of your daily work happens — every controller, model, and request validator you write lives here.

`routes/`

- `routes/web.php` — routes that return Blade views/sessions (browser-based)
- `routes/api.php` — routes that return JSON, stateless (this is what you'll use with React)

`database/`

```
text
```

```
database/
 ├── migrations/   → version-controlled database schema changes
 ├── seeders/      → scripts to insert sample/test data
 └── factories/    → generate fake data for testing
```

`storage/`

```
text

storage/
 ├── app/public/   → uploaded files (images, PDFs)
 ├── framework/    → cache, sessions, compiled views
 └── logs/         → laravel.log — your app's error log
```

When you use it: Whenever you debug an error, check `storage/logs/laravel.log` first.

`public/`

The **only** folder the web server points to. Contains `index.php`, the single entry point for every request, plus compiled CSS/JS and uploaded file symlinks.

PART 5 — ROUTING

Simple explanation: A route connects a URL to a piece of code. In `routes/api.php`:

```
php

use App\Http\Controllers\ProductController;
use Illuminate\Support\Facades\Route;

Route::get('/products', [ProductController::class, 'index']);
Route::post('/products', [ProductController::class, 'store']);
Route::put('/products/{id}', [ProductController::class, 'update']);
Route::patch('/products/{id}', [ProductController::class, 'update']);
Route::delete('/products/{id}', [ProductController::class, 'destroy']);
```

Explanation:

- `Route::get()` handles reading data.
- `Route::post()` handles creating data.
- `Route::put()`/`patch()` handle updating (`put` replaces the whole resource, `patch` updates part of it).
- `Route::delete()` handles deleting.

- `[ProductController::class, 'index']` means "run the `index` method inside `ProductController`."

5.1 Route Parameters

```php
Route::get('/products/{id}', function ($id) {
    return "Product ID: $id";
});
```

5.2 Optional Parameters

```php
Route::get('/products/{id?}', function ($id = null) {
    return $id ? "Product $id" : "All products";
});
```

5.3 Named Routes

```php
Route::get('/products/{id}', [ProductController::class, 'show'])->name('products.show'
```

Why: Lets you generate URLs by name instead of hardcoding paths — `route('products.show', ['id' => 5])`.

5.4 Route Groups & Prefix

```php
Route::prefix('admin')->group(function () {
    Route::get('/products', [AdminProductController::class, 'index']);
    // becomes: /admin/products
});
```

5.5 Route Middleware

```php
```

```php
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/user', function (Request $request) {
        return $request->user();
    });
});
```

Simple explanation: Everything inside this group requires a valid Sanctum token before it runs.

5.6 Route Model Binding

```php
Route::get('/products/{product}', [ProductController::class, 'show']);

// In the controller:
public function show(Product $product) {
    return $product;
}
```

Simple explanation: Laravel automatically finds the `Product` matching the `{product}` ID in the URL and injects the full model — no manual `find()` needed. If not found, it automatically returns a 404.

5.7 API Routes

All routes in `routes/api.php` are automatically prefixed with `/api` and are stateless (no cookies/sessions) — exactly what you want for a React frontend.

PART 6 — CONTROLLERS

Simple explanation: A Controller is a class that groups related request-handling logic together, keeping `routes/api.php` clean.

6.1 Creating a Controller

```bash
php artisan make:controller ProductController --api
```

Explanation: `--api` generates a controller pre-filled with the 5 standard REST methods (`index, store, show, update, destroy`) and skips the Blade-only ones (`create, edit`).

6.2 Full Example Controller

```php
```

```php
<?php

namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    // GET /api/products
    public function index()
    {
        $products = Product::all();
        return response()->json($products);
    }

    // POST /api/products
    public function store(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required|string|max:255',
            'price' => 'required|numeric|min:0',
        ]);

        $product = Product::create($validated);

        return response()->json($product, 201);
    }

    // GET /api/products/{id}
    public function show(Product $product)
    {
        return response()->json($product);
    }

    // PUT /api/products/{id}
    public function update(Request $request, Product $product)
    {
        $validated = $request->validate([
            'name' => 'sometimes|string|max:255',
            'price' => 'sometimes|numeric|min:0',
        ]);

        $product->update($validated);
```

```
        return response()->json($product);
    }

    // DELETE /api/products/{id}
    public function destroy(Product $product)
    {
        $product->delete();
        return response()->json(['message' => 'Product deleted'], 200);
    }
}
```

Explanation of key lines:

- `Request $request` — Laravel automatically injects the incoming HTTP request object (**dependency injection**: Laravel supplies needed objects to a method automatically, instead of you creating them manually).
- `$request->validate([...])` — validates incoming data and throws a 422 error automatically if it fails.
- `response()->json($data, $statusCode)` — builds a JSON HTTP response with a status code.
- `Product $product` in `show`/`update`/`destroy` — this is route model binding from Part 5.6.

6.3 Resource Controllers & Routes

Instead of writing 5 separate `Route::` lines, use:

```php
Route::apiResource('products', ProductController::class);
```

This single line creates all 5 REST routes automatically, matching the controller methods above.

PART 7 — BLADE

Even though your main goal is APIs consumed by React, Laravel still uses Blade for emails, admin panels, and PDF templates, so it's worth knowing the basics.

7.1 What is Blade?

Simple explanation: Blade is Laravel's templating language for generating HTML. It's similar to JSX in spirit (mixing logic with markup) but compiles to plain PHP.

7.2 Variables

blade

```blade
<h1>{{ $title }}</h1>
```

`{{ }}` automatically escapes output (prevents XSS attacks), similar to how React escapes `{variable}` by default.

7.3 Conditions

blade

```blade
@if ($user->isAdmin())
    <p>Welcome, Admin</p>
@else
    <p>Welcome, User</p>
@endif
```

7.4 Loops

blade

```blade
@foreach ($products as $product)
    <li>{{ $product->name }}</li>
@endforeach
```

7.5 Layouts

blade

```blade
{{-- layouts/app.blade.php --}}
<html>
<body>
    @yield('content')
</body>
</html>

{{-- home.blade.php --}}
@extends('layouts.app')
@section('content')
    <h1>Home Page</h1>
@endsection
```

Simple explanation: `@yield` marks a placeholder slot; child views fill it with `@section`. This is like a React layout component wrapping `{children}`.

7.6 Components

blade

```
{{-- components/alert.blade.php --}}
<div class="alert">{{ $slot }}</div>

{{-- usage --}}
<x-alert>Something went wrong!</x-alert>
```

Directly comparable to a reusable React component.

7.7 Forms & CSRF

blade

```
<form method="POST" action="/products">
    @csrf
    <input type="text" name="name">
    <button type="submit">Save</button>
</form>
```

Simple explanation: `@csrf` inserts a hidden token protecting against **Cross-Site Request Forgery** — a security attack where a malicious site tricks a logged-in user's browser into submitting a request. Not needed for pure API routes using Sanctum tokens instead.

7.8 Passing Data from Controller to View

php

```
return view('products.index', ['products' => $products]);
```

7.9 Includes

blade

```
@include('partials.navbar')
```

7.10 Blade vs React

Blade	React
Renders on the **server**	Renders in the **browser**
Returns final HTML	Returns a virtual DOM tree, then real DOM
Good for simple pages, emails, PDFs	Good for interactive SPAs
No client-side state management	Has hooks/state (`useState`, etc.)

In your architecture: Since React is your frontend, Laravel will mostly return **JSON**, not Blade HTML — except for things like password-reset emails, which are still rendered with Blade.

PART 8 — DATABASE FUNDAMENTALS

8.1 Core Concepts

Term	Simple Meaning
Database	A container that stores all your tables
Table	A grid of data, like a spreadsheet (e.g. `products`)
Row	One single record (e.g. one product)
Column	One field/attribute (e.g. `price`)
Primary Key	A unique ID identifying each row (usually `id`)
Foreign Key	A column that points to another table's primary key, creating a relationship
Index	A speed-boosting lookup structure for a column, like a book's index
Unique Constraint	A rule preventing duplicate values in a column (e.g. no two users with the same email)
Relationship	How tables connect to each other (e.g. one user has many posts)

8.2 Laravel Database Configuration

In `.env`:

```
env
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=my_app
DB_USERNAME=root
DB_PASSWORD=secret
```

Laravel reads this and connects automatically — you never write raw connection code.

8.3 Supported Databases

Database	`DB_CONNECTION` value	Notes
MySQL	`mysql`	Most common default
PostgreSQL	`pgsql`	Popular, feature-rich
SQLite	`sqlite`	File-based, great for quick local testing
SQL Server	`sqlserver`	Enterprise Windows environments
Oracle	`oracle` (via community driver)	Requires the `yajra/laravel-oci8` package — covered fully in Part 22

Testing your DB connection:

bash

```
php artisan tinker
>>> DB::connection()->getPdo();
```

If this doesn't throw an error, your connection works.

PART 9 — MIGRATIONS

Simple explanation: A migration is a version-controlled file describing a database change — think of it as "Git for your database structure." Instead of manually clicking around in a database tool, you write PHP code that creates/changes tables, and everyone on your team runs the same migrations to get the same database structure.

9.1 Creating a Migration

bash

```
php artisan make:migration create_products_table
```

This generates a timestamped file in `database/migrations/`.

9.2 Creating a Table

php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();                          // auto-increment primary key
            $table->string('name');
            $table->text('description')->nullable();
            $table->decimal('price', 10, 2);
            $table->integer('stock')->default(0);
            $table->foreignId('category_id')->constrained()->onDelete('cascade');
            $table->timestamps();                  // created_at, updated_at
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

Explanation:

- `up()` — runs when you apply the migration (create the table).
- `down()` — runs when you roll it back (undo it).
- `$table->id()` — creates an auto-incrementing `id` primary key column.
- `->nullable()` — allows the column to be empty.
- `->foreignId('category_id')->constrained()` — creates a foreign key linking to the `categories` table's `id`.
- `->onDelete('cascade')` — if the category is deleted, delete its products too.
- `$table->timestamps()` — auto-adds `created_at` and `updated_at` columns.

9.3 Adding / Removing / Changing Columns

bash

```
php artisan make:migration add_sku_to_products_table --table=products
```

php

```
public function up(): void
{
    Schema::table('products', function (Blueprint $table) {
        $table->string('sku')->nullable()->after('name');
    });
}

public function down(): void
{
    Schema::table('products', function (Blueprint $table) {
        $table->dropColumn('sku');
    });
}
```

To **change** an existing column's type, you need the `doctrine/dbal` package (older Laravel versions) or native `change()` (Laravel 11+):

php

```
Schema::table('products', function (Blueprint $table) {
    $table->string('name', 500)->change();
});
```

9.4 Foreign Keys & Indexes

php

```
$table->foreignId('user_id')->constrained()->onDelete('cascade');
$table->index('email');              // speeds up searches on this column
$table->unique('sku');               // prevents duplicate SKUs
```

9.5 Running Migrations

bash

```
php artisan migrate
```

Explanation: Runs every migration that hasn't been run yet, in order.

```
bash

php artisan migrate:rollback
```

Explanation: Undoes the last batch of migrations (runs their `down()` methods).

```
bash

php artisan migrate:refresh
```

Explanation: Rolls back **all** migrations, then re-runs them all — keeps your migration files but rebuilds the DB structure from scratch.

```
bash

php artisan migrate:fresh
```

Explanation: Drops **all tables** entirely, then re-runs every migration. Fastest way to get a clean slate during development (never run in production).

PART 10 — MODELS & ELOQUENT ORM

10.1 What is a Model?

Simple explanation: A Model is a PHP class representing one database table. `Product` model ↔ `products` table.

10.2 What is an ORM?

Simple explanation: ORM (Object-Relational Mapping) lets you interact with your database using PHP objects and methods instead of writing raw SQL. `Product::find(1)` instead of `SELECT * FROM products WHERE id = 1`.

10.3 What is Eloquent?

Simple explanation: Eloquent is Laravel's built-in ORM. It's often compared to Mongoose (for MongoDB) — but for relational (SQL) databases.

10.4 Creating a Model

```
bash
```

```
php artisan make:model Product -m
```

Explanation: `-m` also generates the matching migration file at the same time.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    protected $fillable = ['name', 'description', 'price', 'stock', 'category_id'];
}
```

10.5 Mass Assignment, `$fillable`, `$guarded`

Simple explanation: Mass assignment means creating/updating a model using an entire array at once, e.g. `Product::create($request->all())`. This is dangerous if not controlled — a malicious user could send extra fields (like `is_admin: true`) that shouldn't be settable.

```php
protected $fillable = ['name', 'price']; // ONLY these fields can be mass-assigned
// OR
protected $guarded = ['id'];             // everything EXCEPT these can be mass-assigne
```

Rule of thumb: Use `$fillable` — it's the safer, explicit whitelist approach.

10.6 CRUD with Eloquent

```php
```

```php
// CREATE
$product = Product::create([
    'name' => 'Laptop',
    'price' => 999.99,
]);

// READ
$product = Product::find(1);
$allProducts = Product::all();

// UPDATE
$product = Product::find(1);
$product->update(['price' => 899.99]);

// DELETE
$product = Product::find(1);
$product->delete();
```

10.7 Soft Deletes

Simple explanation: Instead of permanently removing a row, soft delete just marks it as deleted (sets a `deleted_at` timestamp) — the data is still in the database, just hidden from normal queries. Useful for "trash bin" functionality.

```php
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use SoftDeletes;
}
```

Migration needs:

```php
$table->softDeletes(); // adds deleted_at column
```

```php
```

```php
$product->delete();          // soft delete (sets deleted_at)
$product->restore();         // undo it
$product->forceDelete();     // permanently remove
Product::withTrashed()->get(); // include soft-deleted rows
```

PART 11 — ELOQUENT QUERIES

```php
Product::find(1);                       // find by primary key, returns null if not fou
Product::findOrFail(1);                 // same, but throws 404 if not found (best for

Product::where('price', '>', 100)->get();
Product::where('category_id', 3)->first();  // first matching row

Product::all();                         // every row
Product::get();                         // same as all(), but works after a query chain

Product::create([...]);
Product::find(1)->update([...]);
Product::find(1)->delete();

Product::orderBy('price', 'desc')->get();
Product::latest()->get();               // shortcut for orderBy('created_at', 'desc')

Product::count();
Product::sum('price');
Product::avg('price');

Product::whereIn('category_id', [1, 2, 3])->get();
Product::whereBetween('price', [100, 500])->get();
```

11.1 Query Builder (without a Model)

```php
use Illuminate\Support\Facades\DB;

DB::table('products')->where('price', '>', 100)->get();
```

When to use: Use Eloquent for anything tied to a Model (99% of the time). Use the raw Query Builder for quick, one-off queries or complex reports where you don't need a full

Model.

11.2 Chaining Example

```php
$products = Product::where('stock', '>', 0)
    ->where('category_id', 2)
    ->orderBy('price', 'asc')
    ->take(10)
    ->get();
```

Explanation: Each method returns a "query builder" object, so you can keep chaining conditions before finally calling `get()` (or `first()`, `count()`, etc.) to actually run the query.

PART 12 — DATABASE RELATIONSHIPS

12.1 One-to-One

```text
User
 ↓
Profile
```

Migration (`profiles` table):

```php
$table->foreignId('user_id')->constrained()->onDelete('cascade');
```

Model:

```php
```

```php
class User extends Model {
    public function profile() {
        return $this->hasOne(Profile::class);
    }
}

class Profile extends Model {
    public function user() {
        return $this->belongsTo(User::class);
    }
}
```

Usage:

```php
$user->profile;          // gets the related Profile
$profile->user;          // gets the owning User
```

12.2 One-to-Many

```text
User
 ↓
Posts
```

Migration (`posts` table):

```php
$table->foreignId('user_id')->constrained()->onDelete('cascade');
```

Model:

```php
```

```php
class User extends Model {
    public function posts() {
        return $this->hasMany(Post::class);
    }
}

class Post extends Model {
    public function user() {
        return $this->belongsTo(User::class);
    }
}
```

Usage:

```php
$user->posts;             // all posts belonging to this user
$post->user;               // the author
```

12.3 Many-to-Many

```text
Students
   ↕
Courses
```

Requires a **pivot table** — a middle table connecting both sides.

Migration (`course_student` table, alphabetical singular names):

```php
Schema::create('course_student', function (Blueprint $table) {
    $table->id();
    $table->foreignId('course_id')->constrained()->onDelete('cascade');
    $table->foreignId('student_id')->constrained()->onDelete('cascade');
    $table->timestamps();
});
```

Model:

```php
```

```php
class Student extends Model {
    public function courses() {
        return $this->belongsToMany(Course::class);
    }
}

class Course extends Model {
    public function students() {
        return $this->belongsToMany(Student::class);
    }
}
```

Usage:

```php
$student->courses;                          // all courses this student takes
$student->courses()->attach($courseId);     // enroll in a course
$student->courses()->detach($courseId);     // unenroll
$student->courses()->sync([1, 2, 3]);       // set exact list of courses
```

12.4 Eager Loading vs Lazy Loading

Simple explanation:

- **Lazy loading** = related data is fetched only when accessed, one query at a time. Causes the "N+1 problem" (1 query for products + N queries, one per product, for their category — very slow).
- **Eager loading** = related data is fetched upfront in one extra query, using `with()`.

```php
// BAD (N+1 problem): triggers 1 query per product for its category
$products = Product::all();
foreach ($products as $product) {
    echo $product->category->name;
}

// GOOD: only 2 queries total, no matter how many products
$products = Product::with('category')->get();
foreach ($products as $product) {
    echo $product->category->name;
}
```

12.5 `whereHas()` — Filtering by a Relationship

```php
// Get all users who have at least one post
$users = User::whereHas('posts')->get();

// Get all users who have a post with more than 100 views
$users = User::whereHas('posts', function ($query) {
    $query->where('views', '>', 100);
})->get();
```

12.6 Nested Relationships

```php
$orders = Order::with('items.product')->get();
```

Explanation: Loads each `Order`, its `items`, and each item's `product`, all in a small, fixed number of queries — no matter how many orders exist.

PART 13 — CRUD API (COMPLETE PRODUCT EXAMPLE)

Let's build a fully working Product CRUD API end-to-end.

13.1 Migration

```bash
php artisan make:model Product -m
```

`database/migrations/xxxx_create_products_table.php`:

```php
```

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->text('description')->nullable();
            $table->decimal('price', 10, 2);
            $table->integer('stock')->default(0);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

Run it:

```bash
php artisan migrate
```

13.2 Model

`app/Models/Product.php`:

```php
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    protected $fillable = ['name', 'description', 'price', 'stock'];
}
```

13.3 Controller

bash

```bash
php artisan make:controller ProductController --api
```

`app/Http/Controllers/ProductController.php`:

php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index()
    {
        return response()->json(Product::latest()->get());
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required|string|max:255',
            'description' => 'nullable|string',
            'price' => 'required|numeric|min:0',
            'stock' => 'required|integer|min:0',
        ]);

        $product = Product::create($validated);

        return response()->json([
            'message' => 'Product created successfully',
            'data' => $product,
        ], 201);
    }

    public function show(Product $product)
    {
        return response()->json($product);
    }

    public function update(Request $request, Product $product)
    {
        $validated = $request->validate([
            'name' => 'sometimes|required|string|max:255',
            'description' => 'nullable|string',
            'price' => 'sometimes|required|numeric|min:0',
            'stock' => 'sometimes|required|integer|min:0',
        ]);
```

```
        $product->update($validated);

        return response()->json([
            'message' => 'Product updated successfully',
            'data' => $product,
        ]);
    }

    public function destroy(Product $product)
    {
        $product->delete();

        return response()->json([
            'message' => 'Product deleted successfully',
        ]);
    }
}
```

13.4 Routes

`routes/api.php`:

```php
<?php

use App\Http\Controllers\ProductController;
use Illuminate\Support\Facades\Route;

Route::apiResource('products', ProductController::class);
```

13.5 Testing with Real Requests

```text
POST   /api/products      { "name": "Laptop", "price": 999.99, "stock": 5 }
GET    /api/products
GET    /api/products/1
PUT    /api/products/1    { "price": 899.99 }
DELETE /api/products/1
```

You now have a complete, working REST API.

PART 14 — FORM REQUEST VALIDATION

Inline `$request->validate()` is fine for small controllers, but professional projects extract validation into **Form Request** classes to keep controllers clean.

14.1 Creating a Form Request

bash

```
php artisan make:request StoreProductRequest
```

`app/Http/Requests/StoreProductRequest.php`:

php

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreProductRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true; // set logic here if only certain users can make this request
    }

    public function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'description' => 'nullable|string',
            'price' => 'required|numeric|min:0',
            'stock' => 'required|integer|min:0',
            'category_id' => 'required|exists:categories,id',
            'sku' => 'nullable|string|unique:products,sku',
        ];
    }

    public function messages(): array
    {
        return [
            'name.required' => 'Product name is required.',
            'price.min' => 'Price cannot be negative.',
        ];
    }
}
```

14.2 Using It in the Controller

php

```php
public function store(StoreProductRequest $request)
{
    $product = Product::create($request->validated());
    return response()->json($product, 201);
}
```

Explanation: Laravel validates the request **before** the controller method even runs. If validation fails, Laravel automatically returns a `422 Unprocessable Entity` response with error details — you don't write any `if` checks yourself.

14.3 Common Validation Rules

Rule	Meaning
`required`	field must be present and not empty
`string`	must be a string
`email`	must be a valid email format
`integer`	must be a whole number
`min:3` / `max:255`	length or value boundaries
`unique:products,sku`	must not already exist in that column
`exists:categories,id`	must exist as an `id` in the `categories` table
`nullable`	field is optional
`sometimes`	only validate if the field is present in the request

PART 15 — API RESOURCES

15.1 What is an API Resource?

Simple explanation: An API Resource is a transformation layer that controls exactly what JSON shape gets sent to the frontend — instead of returning the raw Eloquent model (which might expose sensitive fields).

15.2 Why Use It?

- Hide sensitive fields (like `password`)
- Rename fields for the frontend
- Add computed/derived fields
- Keep a consistent JSON structure across the whole API

15.3 Creating One

bash

```
php artisan make:resource ProductResource
```

`app/Http/Resources/ProductResource.php`:

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class ProductResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'price' => (float) $this->price,
            'in_stock' => $this->stock > 0,
            'category' => $this->whenLoaded('category', fn() => $this->category->name),
            'created_at' => $this->created_at->toDateTimeString(),
        ];
    }
}
```

Explanation:

- `$this->column` accesses the underlying model's attribute.
- `whenLoaded('category', ...)` only includes the category if it was eager-loaded — avoids accidental extra queries.

15.4 Using It

```php
// single model
return new ProductResource($product);

// collection
return ProductResource::collection(Product::all());
```

15.5 Nested Resources

```php
class OrderResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'items' => OrderItemResource::collection($this->whenLoaded('items')),
        ];
    }
}
```

PART 16 — API PAGINATION

16.1 Why Paginate?

Returning 10,000 products in one response is slow and wasteful. Pagination splits results into pages.

```php
public function index()
{
    $products = Product::paginate(10); // 10 per page
    return response()->json($products);
}
```

Request:

```text
GET /api/products?page=2
```

16.2 Custom Page Size

```php
$perPage = request('per_page', 10); // default 10 if not provided
$products = Product::paginate($perPage);
```

Request:

text

```
GET /api/products?page=2&per_page=20
```

16.3 Response Shape

json

```
{
    "current_page": 2,
    "data": [ ... ],
    "first_page_url": "...?page=1",
    "last_page": 8,
    "next_page_url": "...?page=3",
    "prev_page_url": "...?page=1",
    "per_page": 10,
    "total": 78
}
```

Explanation: Laravel automatically builds this metadata — `total` (all matching rows), `last_page` (total pages available), and ready-to-use `next_page_url`/`prev_page_url` links your React app can call directly.

PART 17 — SEARCH, FILTER & SORT

Combine all three into one professional endpoint:

php

```php
public function index(Request $request)
{
    $query = Product::query();

    if ($request->filled('search')) {
        $query->where('name', 'like', '%' . $request->search . '%');
    }

    if ($request->filled('category')) {
        $query->whereHas('category', function ($q) use ($request) {
            $q->where('slug', $request->category);
        });
    }

    if ($request->filled('min_price')) {
        $query->where('price', '>=', $request->min_price);
    }

    if ($request->filled('max_price')) {
        $query->where('price', '<=', $request->max_price);
    }

    $sort = $request->get('sort', 'created_at');
    $order = $request->get('order', 'desc');
    $query->orderBy($sort, $order);

    return response()->json($query->paginate($request->get('per_page', 10)));
}
```

Simple explanation of the flow:

1. Start with a base query (`Product::query()` — doesn't run yet, just builds).
2. `$request->filled('search')` checks if that query parameter was sent and isn't empty.
3. Each `if` block adds another `where` condition, only when relevant.
4. Sorting is applied last, using safe defaults if not provided.
5. Finally, paginate and return.

Example request:

```text
GET /api/products?search=laptop&category=electronics&min_price=500&max_price=2000&sort=
```

PART 18 — AUTHENTICATION (LARAVEL SANCTUM)

Simple explanation of terms:

- **Authentication** = confirming who a user is (login).
- **Password hashing** = scrambling a password into an unreadable string before storing it, so even if the database leaks, real passwords stay hidden.
- **Sanctum** = Laravel's official lightweight package for issuing API tokens — perfect for a React SPA talking to a Laravel API.

18.1 Installing Sanctum

bash

```
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate
```

In `app/Models/User.php`:

php

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
}
```

18.2 AuthController — Complete Implementation

bash

```
php artisan make:controller AuthController
```

`app/Http/Controllers/AuthController.php`:

php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthController extends Controller
{
    public function register(Request $request)
    {
        $validated = $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|string|min:8|confirmed',
        ]);

        $user = User::create([
            'name' => $validated['name'],
            'email' => $validated['email'],
            'password' => Hash::make($validated['password']),
        ]);

        $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json([
            'user' => $user,
            'token' => $token,
        ], 201);
    }

    public function login(Request $request)
    {
        $request->validate([
            'email' => 'required|email',
            'password' => 'required|string',
        ]);

        $user = User::where('email', $request->email)->first();

        if (!$user || !Hash::check($request->password, $user->password)) {
            throw ValidationException::withMessages([
```

```
                'email' => ['The provided credentials are incorrect.'],
            ]);
        }

        $token = $user->createToken('auth_token')->plainTextToken;

        return response()->json([
            'user' => $user,
            'token' => $token,
        ]);
    }

    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();

        return response()->json(['message' => 'Logged out successfully']);
    }

    public function user(Request $request)
    {
        return response()->json($request->user());
    }
}
```

Explanation of key lines:

- `Hash::make()` — hashes the password using bcrypt before saving. Never store plain-text passwords.
- `Hash::check($plain, $hashed)` — compares a plain-text password against the stored hash.
- `confirmed` rule — requires a matching `password_confirmation` field, same as a "confirm password" field in a signup form.
- `createToken('auth_token')->plainTextToken` — generates an API token; React stores this and sends it on every future request.
- `$request->user()` — Sanctum automatically resolves the currently authenticated user from the token.

18.3 Routes

```
php
```

```php
use App\Http\Controllers\AuthController;

Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/user', [AuthController::class, 'user']);
});
```

18.4 How React Sends the Token

```javascript
axios.get('/api/user', {
    headers: {
        Authorization: `Bearer ${token}`
    }
});
```

Simple explanation: Every protected request must include this header. Laravel's `auth:sanctum` middleware reads it, finds the matching user, and injects them into `$request->user()`.

PART 19 — AUTHORIZATION

```text
Authentication = Who are you?
Authorization  = What are you allowed to do?
```

19.1 Middleware-Based Authorization (Simple)

```php
Route::middleware(['auth:sanctum', 'role:admin'])->group(function () {
    Route::delete('/products/{product}', [ProductController::class, 'destroy']);
});
```

(Requires the custom `role` middleware built in Part 23.)

19.2 Gates

Simple explanation: A Gate is a simple yes/no rule defined as a closure, good for actions not tied to one specific model.

`app/Providers/AppServiceProvider.php`:

```php
use Illuminate\Support\Facades\Gate;

public function boot(): void
{
    Gate::define('access-admin-panel', function ($user) {
        return $user->role === 'admin';
    });
}
```

```php
if (Gate::allows('access-admin-panel')) {
    // allowed
}
```

19.3 Policies

Simple explanation: A Policy is a class dedicated to authorization rules for one specific Model — the recommended approach for anything CRUD-related.

```bash
php artisan make:policy ProductPolicy --model=Product
```

`app/Policies/ProductPolicy.php`:

```php
```

```php
<?php

namespace App\Policies;

use App\Models\Product;
use App\Models\User;

class ProductPolicy
{
    public function update(User $user, Product $product): bool
    {
        return $user->role === 'admin';
    }

    public function delete(User $user, Product $product): bool
    {
        return $user->role === 'admin';
    }
}
```

Usage in a controller:

```php
public function update(Request $request, Product $product)
{
    $this->authorize('update', $product);
    // continues only if allowed, otherwise throws 403 automatically
    $product->update($request->validated());
    return response()->json($product);
}
```

19.4 Roles & Permissions Example

```php
// migration: add role column
$table->string('role')->default('customer'); // admin | teacher | student | customer
```

```php
```

```php
class User extends Authenticatable
{
    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }
}
```

```text
Admin    → full access to everything
Teacher  → manage own courses only
Student  → read-only access, enroll in courses
```

PART 20 — FILE & IMAGE UPLOAD

20.1 Key Concepts

- **Multipart/form-data** = the HTTP encoding format required to send files (not plain JSON).
- **Laravel Storage** = an abstraction over where files are actually saved (local disk, S3, etc.) — you write the same code regardless of the underlying storage.
- **Public disk** = a storage location whose files are accessible via a public URL.

20.2 Setup

```bash
php artisan storage:link
```

Explanation: Creates a symbolic link from `public/storage` → `storage/app/public`, making uploaded files reachable by URL.

20.3 Controller Example

```php
```

```php
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'price' => 'required|numeric|min:0',
        'image' => 'nullable|image|mimes:jpg,jpeg,png,webp|max:2048', // max 2MB
    ]);

    if ($request->hasFile('image')) {
        $path = $request->file('image')->store('products', 'public');
        $validated['image'] = $path;
    }

    $product = Product::create($validated);

    return response()->json($product, 201);
}
```

Explanation:

- `image|mimes:...` — validates it's an actual image of an allowed type.
- `->store('products', 'public')` — saves the file under `storage/app/public/products/`, returns the relative path.

20.4 Generating a Full URL

```php
// in the model or a resource
'image_url' => $this->image ? asset('storage/' . $this->image) : null,
```

20.5 Deleting a File

```php
use Illuminate\Support\Facades\Storage;

Storage::disk('public')->delete($product->image);
```

20.6 React → Laravel Upload (FormData)

```javascript
```

```
const formData = new FormData();
formData.append('name', 'Laptop');
formData.append('price', 999.99);
formData.append('image', fileInputRef.current.files[0]);

axios.post('/api/products', formData, {
    headers: {
        'Content-Type': 'multipart/form-data',
        Authorization: `Bearer ${token}`,
    },
});
```

Simple explanation: `FormData` is a browser API that builds a multipart request body — the same format `<form enctype="multipart/form-data">` would send, which is what a file upload requires (plain JSON can't carry binary file data).

PART 21 — LARAVEL + REACT

21.1 Full Architecture

```
text

React (Axios)
   ↓
Laravel API (routes/api.php)
   ↓
Controller
   ↓
Service
   ↓
Eloquent
   ↓
Database
```

21.2 CORS

Simple explanation: CORS (Cross-Origin Resource Sharing) is a browser security rule blocking a website from calling an API on a different domain/port unless that API explicitly allows it. Since React (e.g. `localhost:3000`) and Laravel (e.g. `localhost:8000`) run on different ports, you must configure CORS.

`config/cors.php`:

```
php
```

```
'paths' => ['api/*'],
'allowed_methods' => ['*'],
'allowed_origins' => ['http://localhost:3000'],
'allowed_headers' => ['*'],
'supports_credentials' => true,
```

21.3 Axios Setup in React

javascript

```javascript
// api.js
import axios from 'axios';

const api = axios.create({
    baseURL: 'http://localhost:8000/api',
});

api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token');
    if (token) {
        config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
});

export default api;
```

21.4 Environment Variables (React side)

env

```
# .env
REACT_APP_API_URL=http://localhost:8000/api
```

21.5 Full CRUD Example in React

javascript

```
import { useEffect, useState } from 'react';
import api from './api';

function ProductList() {
    const [products, setProducts] = useState([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        api.get('/products')
            .then((res) => setProducts(res.data.data))
            .catch((err) => setError(err.response?.data?.message || 'Something went wr
            .finally(() => setLoading(false));
    }, []);

    const deleteProduct = async (id) => {
        await api.delete(`/products/${id}`);
        setProducts(products.filter((p) => p.id !== id));
    };

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error}</p>;

    return (
        <ul>
            {products.map((product) => (
                <li key={product.id}>
                    {product.name} — ${product.price}
                    <button onClick={() => deleteProduct(product.id)}>Delete</button>
                </li>
            ))}
        </ul>
    );
}

export default ProductList;
```

Explanation: This mirrors patterns you already know from working with Express APIs — `useEffect` fetches on mount, `loading`/`error` state handle UX, and every write action (`delete`) immediately updates local state instead of re-fetching everything.

21.6 Error Handling Pattern

Laravel validation errors (422) come back shaped like:

```json
{
    "message": "The given data was invalid.",
    "errors": {
        "email": ["The email field is required."]
    }
}
```

React can read `err.response.data.errors` to show field-specific messages.

PART 22 — ORACLE + LARAVEL

22.1 What is Oracle Database?

Simple explanation: Oracle Database is an enterprise-grade relational database, common in banks, telecoms, and large corporations. It's more strict and feature-heavy than MySQL, and historically more expensive/complex to set up.

22.2 Oracle Local Server vs Oracle Cloud

- **Oracle local server (Oracle XE — Express Edition):** free, lightweight version you can install on your machine for learning.
- **Oracle Cloud (Autonomous Database):** a managed, cloud-hosted Oracle database — Oracle offers a free tier good for practice without local installation.

22.3 Oracle Client & OCI8

Simple explanation:

- **Oracle Client** = software libraries that let any application (including PHP) talk to an Oracle database.
- **OCI8** = the official PHP extension that uses the Oracle Client to run queries from PHP code.

You cannot connect PHP to Oracle without both installed and configured correctly — this is the trickiest part of the whole setup.

22.4 Installing the PHP Oracle Driver

1. Download **Oracle Instant Client** (Basic + SDK packages) for your OS from Oracle's official site.
2. Enable the `oci8` extension in `php.ini`:

```ini
extension=oci8
```

3. Verify:

```bash
php -m | grep oci8
```

22.5 Laravel Oracle Connection Package

Laravel has no Oracle driver built in — install the community package:

```bash
composer require yajra/laravel-oci8
```

22.6 `.env` Configuration

```env
DB_CONNECTION=oracle
DB_HOST=127.0.0.1
DB_PORT=1521
DB_DATABASE=XE
DB_USERNAME=system
DB_PASSWORD=oracle
DB_SERVICE_NAME=XE
DB_CHARSET=AL32UTF8
```

`config/database.php` (added automatically by the package, but shown for clarity):

```php
'oracle' => [
    'driver' => 'oracle',
    'host' => env('DB_HOST'),
    'port' => env('DB_PORT', '1521'),
    'database' => env('DB_DATABASE'),
    'service_name' => env('DB_SERVICE_NAME'),
    'username' => env('DB_USERNAME'),
    'password' => env('DB_PASSWORD'),
    'charset' => env('DB_CHARSET', 'AL32UTF8'),
    'prefix' => '',
],
```

22.7 Testing the Connection

bash

```
php artisan tinker
>>> DB::connection('oracle')->getPdo();
```

If no error is thrown, the connection works.

22.8 Building CRUD on Oracle

The great news: **your Eloquent code doesn't change at all.** Migrations, models, and controllers work exactly the same — only the `.env` connection changes.

php

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->string('password');
    $table->timestamps();
});

Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->decimal('price', 10, 2);
    $table->timestamps();
});

Schema::create('orders', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained();
    $table->decimal('total', 10, 2);
    $table->timestamps();
});
```

Run:

bash

```
php artisan migrate --database=oracle
```

`Product::create()`, `Order::with('user')->get()`, etc. all work unchanged.

22.9 How Oracle Differs from MySQL

MySQL	Oracle
Auto-increment via `AUTO_INCREMENT`	Uses **sequences** and **triggers** internally (Laravel's package handles this for you)
Table/column names usually lowercase	Traditionally UPPERCASE identifiers (the package normalizes this)
`LIMIT`/`OFFSET` for pagination	Uses `ROWNUM`/`FETCH FIRST` under the hood (again, handled by Eloquent automatically)
Free, open-source	Commercial, enterprise-licensed (Express Edition is free but limited)
Simpler for small/medium apps	Preferred for large enterprise systems needing strict transactions, auditing

Bottom line: Once the connection and package are configured correctly, you write Laravel code exactly the same way regardless of whether the database engine is MySQL or Oracle — that's the entire point of using an ORM.

PART 23 — MIDDLEWARE

23.1 What Middleware Is

Simple explanation: Middleware = a layer that checks (or modifies) a request before it reaches the controller, or a response before it's sent back. Think of Express.js middleware — same exact idea.

23.2 Why Middleware

- Block unauthenticated users
- Check user roles
- Log requests
- Rate-limit requests
- Add CORS headers

23.3 Creating Middleware

```bash
php artisan make:middleware EnsureUserIsAdmin
```

`app/Http/Middleware/EnsureUserIsAdmin.php`:

php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class EnsureUserIsAdmin
{
    public function handle(Request $request, Closure $next)
    {
        if ($request->user()?->role !== 'admin') {
            return response()->json(['message' => 'Forbidden'], 403);
        }

        return $next($request);
    }
}
```

Explanation: `$next($request)` passes control forward to the next middleware/controller. If you don't call it, the request stops right there — useful for blocking access.

23.4 Registering Middleware (Laravel 11+)

`bootstrap/app.php`:

php

```php
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'admin' => \App\Http\Middleware\EnsureUserIsAdmin::class,
    ]);
})
```

23.5 Middleware with Parameters (Role Middleware)

php

```php
class RoleMiddleware
{
    public function handle(Request $request, Closure $next, string $role)
    {
        if ($request->user()?->role !== $role) {
            return response()->json(['message' => 'Forbidden'], 403);
        }

        return $next($request);
    }
}
```

Usage:

```php
Route::middleware('role:admin')->group(function () {
    // admin-only routes
});
```

23.6 Applying Middleware

```php
// on a single route
Route::get('/admin/dashboard', [DashboardController::class, 'index'])->middleware('adm

// on a group
Route::middleware(['auth:sanctum', 'admin'])->group(function () {
    // ...
});
```

PART 24 — SERVICE CLASSES

24.1 Why Controllers Shouldn't Get Huge

Simple explanation: If you put all your business logic (calculations, external API calls, complex rules) directly in controllers, they become hard to read, test, and reuse. A Service class extracts that logic into its own dedicated class.

```text
```

```
Controller  (handles HTTP concerns: request/response)
   ↓
Service     (handles business logic)
   ↓
Model       (handles database access)
```

24.2 Example: ProductService

bash

```bash
mkdir -p app/Services
```

`app/Services/ProductService.php`:

php

```php
<?php

namespace App\Services;

use App\Models\Product;
use Illuminate\Support\Facades\Storage;

class ProductService
{
    public function create(array $data, ?\Illuminate\Http\UploadedFile $image = null):
    {
        if ($image) {
            $data['image'] = $image->store('products', 'public');
        }

        return Product::create($data);
    }

    public function update(Product $product, array $data, ?\Illuminate\Http\UploadedFi
    {
        if ($image) {
            if ($product->image) {
                Storage::disk('public')->delete($product->image);
            }
            $data['image'] = $image->store('products', 'public');
        }

        $product->update($data);
        return $product;
    }

    public function delete(Product $product): void
    {
        if ($product->image) {
            Storage::disk('public')->delete($product->image);
        }
        $product->delete();
    }
}
```

24.3 Using It in a Controller

php

```php
class ProductController extends Controller
{
    public function __construct(protected ProductService $productService) {}

    public function store(StoreProductRequest $request)
    {
        $product = $this->productService->create(
            $request->validated(),
            $request->file('image')
        );

        return response()->json($product, 201);
    }
}
```

Explanation: `protected ProductService $productService` in the constructor uses **dependency injection** — Laravel automatically creates and provides a `ProductService` instance for you, no manual `new ProductService()` needed.

24.4 When Services Are Useful

- Logic reused across multiple controllers
- Complex business rules (e.g. "when an order is placed, reduce stock, send email, create invoice")
- Anything involving multiple models working together

When NOT needed: Simple CRUD with no extra logic — keeping it directly in the controller is fine and avoids over-engineering.

PART 25 — REPOSITORY PATTERN

25.1 What Repository Means

Simple explanation: A Repository is a class that isolates all database-query logic behind a consistent interface, so the rest of your app doesn't know or care whether data comes from Eloquent, an API, or a cache.

```text
```

```
Controller
 ↓
Service
 ↓
Repository    ← all Eloquent queries live here
 ↓
Model
 ↓
Database
```

25.2 Why Repositories Are Sometimes Useful

- Swapping data sources without touching business logic
- Centralizing complex queries in one place
- Easier to mock in tests

25.3 Repository Interface

`app/Repositories/ProductRepositoryInterface.php`:

php

```php
<?php

namespace App\Repositories;

use App\Models\Product;
use Illuminate\Support\Collection;

interface ProductRepositoryInterface
{
    public function all(): Collection;
    public function find(int $id): ?Product;
    public function create(array $data): Product;
    public function update(Product $product, array $data): Product;
    public function delete(Product $product): void;
}
```

25.4 Repository Implementation

`app/Repositories/EloquentProductRepository.php`:

php

```php
<?php

namespace App\Repositories;

use App\Models\Product;
use Illuminate\Support\Collection;

class EloquentProductRepository implements ProductRepositoryInterface
{
    public function all(): Collection
    {
        return Product::all();
    }

    public function find(int $id): ?Product
    {
        return Product::find($id);
    }

    public function create(array $data): Product
    {
        return Product::create($data);
    }

    public function update(Product $product, array $data): Product
    {
        $product->update($data);
        return $product;
    }

    public function delete(Product $product): void
    {
        $product->delete();
    }
}
```

25.5 Binding the Interface

`app/Providers/AppServiceProvider.php`:

```php
```

```
public function register(): void
{
    $this->app->bind(
        \App\Repositories\ProductRepositoryInterface::class,
        \App\Repositories\EloquentProductRepository::class
    );
}
```

25.6 Important Warning

> **Do NOT use the Repository Pattern everywhere unnecessarily.** For a small-to-medium Laravel app, Eloquent models already act as a clean abstraction — adding a Repository on top of every single model adds boilerplate without real benefit. Reserve it for large, complex projects where you genuinely need to decouple your data layer (e.g. supporting multiple databases, heavy caching strategies, or extensive testing needs).

PART 26 — EVENTS & LISTENERS

26.1 What Events & Listeners Are

Simple explanation:

- **Event** = an announcement that "something happened" (e.g. "a user registered").
- **Listener** = code that reacts to that announcement (e.g. "send a welcome email").

This decouples unrelated actions — your registration logic doesn't need to know anything about email sending.

26.2 Event Flow

```
text

User registers
   ↓
UserRegistered event fires
   ↓
Listener(s) react
   ↓
SendWelcomeEmail listener sends the email
```

26.3 Creating an Event & Listener

```
bash
```

```
php artisan make:event UserRegistered
php artisan make:listener SendWelcomeEmail --event=UserRegistered
```

`app/Events/UserRegistered.php`:

```php
<?php

namespace App\Events;

use App\Models\User;
use Illuminate\Foundation\Events\Dispatchable;

class UserRegistered
{
    use Dispatchable;

    public function __construct(public User $user) {}
}
```

`app/Listeners/SendWelcomeEmail.php`:

```php
<?php

namespace App\Listeners;

use App\Events\UserRegistered;
use App\Mail\WelcomeMail;
use Illuminate\Support\Facades\Mail;

class SendWelcomeEmail
{
    public function handle(UserRegistered $event): void
    {
        Mail::to($event->user->email)->send(new WelcomeMail($event->user));
    }
}
```

26.4 Firing the Event

```php
use App\Events\UserRegistered;

$user = User::create($validated);
UserRegistered::dispatch($user);
```

26.5 Real-World Use Cases

- Order placed → reduce stock, send confirmation email, notify admin
- User registered → send welcome email, create default profile
- Product low on stock → notify admin

PART 27 — JOBS & QUEUES

27.1 The Problem

Simple explanation: Some tasks are slow (sending an email, generating a PDF, calling a third-party API). If you run them directly inside a request, the user waits (or worse, the request times out). **Queues** let you run these tasks in the background instead.

27.2 Jobs

```bash
php artisan make:job SendOrderConfirmationEmail
```

`app/Jobs/SendOrderConfirmationEmail.php`:

```php
```

```php
<?php

namespace App\Jobs;

use App\Mail\OrderConfirmationMail;
use App\Models\Order;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Mail;

class SendOrderConfirmationEmail implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public int $tries = 3; // retry up to 3 times if it fails

    public function __construct(public Order $order) {}

    public function handle(): void
    {
        Mail::to($this->order->user->email)->send(new OrderConfirmationMail($this->ord
    }
}
```

27.3 Dispatching a Job

```php
SendOrderConfirmationEmail::dispatch($order);
```

Explanation: This doesn't run immediately — it's pushed onto a queue and processed separately, so the user's request returns instantly.

27.4 Queue Configuration

`.env`:

```env
QUEUE_CONNECTION=database   # or redis in production
```

```bash
php artisan queue:table
php artisan migrate
```

27.5 Running a Worker

```bash
php artisan queue:work
```

Simple explanation: A **worker** is a background process constantly checking the queue for new jobs and running them one by one. Without a running worker, dispatched jobs just sit waiting.

27.6 Failed Jobs & Retry

```bash
php artisan queue:failed          # list failed jobs
php artisan queue:retry all       # retry every failed job
php artisan queue:retry {id}      # retry a specific one
```

`public int $tries = 3;` on the job class controls automatic retry attempts before it's marked as permanently failed.

PART 28 — MAIL

28.1 SMTP & Mail Configuration

Simple explanation: SMTP (Simple Mail Transfer Protocol) is the standard protocol used to send emails. Laravel needs SMTP credentials to actually deliver mail (for local dev, tools like Mailtrap catch emails without really sending them).

`.env`:

```env
```

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.mailtrap.io
MAIL_PORT=2525
MAIL_USERNAME=your_username
MAIL_PASSWORD=your_password
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@myapp.com
MAIL_FROM_NAME="My App"
```

28.2 Creating a Mailable

bash

```
php artisan make:mail WelcomeMail
```

`app/Mail/WelcomeMail.php`:

php

```
<?php

namespace App\Mail;

use App\Models\User;
use Illuminate\Mail\Mailable;
use Illuminate\Queue\SerializesModels;

class WelcomeMail extends Mailable
{
    use SerializesModels;

    public function __construct(public User $user) {}

    public function build()
    {
        return $this->subject('Welcome to My App!')
                    ->view('emails.welcome')
                    ->with(['name' => $this->user->name]);
    }
}
```

28.3 Email Template (Blade)

`resources/views/emails/welcome.blade.php`:

```blade
<h1>Welcome, {{ $name }}!</h1>
<p>Thanks for joining My App. We're excited to have you.</p>
```

28.4 Sending an Email

```php
use Illuminate\Support\Facades\Mail;
use App\Mail\WelcomeMail;

Mail::to($user->email)->send(new WelcomeMail($user));
```

Best practice: Wrap this in a Job (Part 27) so email sending never slows down the user's request.

28.5 Attachments

```php
public function build()
{
    return $this->subject('Your Invoice')
                ->view('emails.invoice')
                ->attach(storage_path('app/invoices/invoice-123.pdf'), [
                    'as' => 'invoice.pdf',
                    'mime' => 'application/pdf',
                ]);
}
```

PART 29 — NOTIFICATIONS

29.1 What Notifications Are

Simple explanation: Notifications are a unified way to alert a user through multiple channels (email, database, SMS, Slack) using one class, instead of writing separate logic for each channel.

29.2 Creating a Notification

```bash
php artisan make:notification OrderShipped
```

`app/Notifications/OrderShipped.php`:

```php
<?php

namespace App\Notifications;

use App\Models\Order;
use Illuminate\Bus\Queueable;
use Illuminate\Notifications\Messages\MailMessage;
use Illuminate\Notifications\Notification;

class OrderShipped extends Notification
{
    use Queueable;

    public function __construct(public Order $order) {}

    public function via($notifiable): array
    {
        return ['mail', 'database'];
    }

    public function toMail($notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject('Your Order Has Shipped')
            ->line("Order #{$this->order->id} is on its way!")
            ->action('Track Order', url("/orders/{$this->order->id}"));
    }

    public function toArray($notifiable): array
    {
        return [
            'order_id' => $this->order->id,
            'message' => 'Your order has shipped.',
        ];
    }
}
```

29.3 Sending It

```php
```

```php
$user->notify(new OrderShipped($order));
```

29.4 Database Notifications Table

bash

```bash
php artisan notifications:table
php artisan migrate
```

php

```php
$user->unreadNotifications;          // get unread notifications
$user->notifications;                // all notifications
$notification->markAsRead();
```

29.5 Notification Channels Summary

Channel	Use case
`mail`	Email alerts
`database`	In-app notification bell/list
`broadcast`	Real-time via WebSockets (e.g. Pusher/Laravel Echo)
`sms` (via 3rd-party like Vonage)	Text message alerts

PART 30 — ERROR HANDLING

30.1 Standard API Error Response

Consistency matters — every error your API returns should follow the same shape so React can handle it predictably:

json

```json
{
    "success": false,
    "message": "Product not found",
    "data": null
}
```

30.2 Custom Exception Handling (Laravel 11+)

`bootstrap/app.php`:

php

```php
->withExceptions(function (Exceptions $exceptions) {
    $exceptions->render(function (\Illuminate\Database\Eloquent\ModelNotFoundException
        if ($request->is('api/*')) {
            return response()->json([
                'success' => false,
                'message' => 'Resource not found',
                'data' => null,
            ], 404);
        }
    });

    $exceptions->render(function (\Illuminate\Validation\ValidationException $e, $requ
        if ($request->is('api/*')) {
            return response()->json([
                'success' => false,
                'message' => 'Validation failed',
                'errors' => $e->errors(),
            ], 422);
        }
    });
})
```

30.3 Custom Exceptions

bash

```bash
php artisan make:exception OutOfStockException
```

php

```php
<?php

namespace App\Exceptions;

use Exception;

class OutOfStockException extends Exception
{
    public function render($request)
    {
        return response()->json([
            'success' => false,
            'message' => 'This product is out of stock',
            'data' => null,
        ], 400);
    }
}
```

Throwing it:

```php
if ($product->stock <= 0) {
    throw new OutOfStockException();
}
```

30.4 Logging

```php
use Illuminate\Support\Facades\Log;

Log::info('Order created', ['order_id' => $order->id]);
Log::error('Payment failed', ['error' => $e->getMessage()]);
```

Logs are written to `storage/logs/laravel.log` — always your first stop when debugging a production issue.

30.5 Debugging Tools

- `APP_DEBUG=true` in `.env` (local only!) shows detailed stack traces in responses.
- `dd($variable)` — "dump and die," prints a variable and stops execution (Laravel's version of `console.log()` + early return).
- `dump($variable)` — prints without stopping execution.

- **Laravel Telescope** — a dashboard showing every request, query, job, and exception during local development.

PART 31 — SECURITY

Concern	Simple Explanation	How Laravel Handles It
Password hashing	Never store plain passwords	`Hash::make()` (bcrypt/argon2) built in
CSRF	Attacker tricks a logged-in browser into submitting a request	`@csrf` token for web forms (not needed for token-based API auth)
CORS	Controls which frontend domains can call your API	`config/cors.php`
SQL Injection	Attacker sneaks SQL into user input to manipulate your database	Eloquent/Query Builder auto-escapes all bound parameters
XSS (Cross-Site Scripting)	Attacker injects malicious JS into pages other users view	Blade's `{{ }}` auto-escapes output; React also escapes by default
Mass assignment	Attacker sends extra unexpected fields in a request	`$fillable` whitelist (Part 10.5)
Authentication	Confirming identity	Sanctum tokens
Authorization	Confirming permission	Policies/Gates
Rate limiting	Prevents abuse (too many requests too fast)	`throttle:60,1` middleware (60 requests per minute)
Validation	Reject malformed/malicious input	Form Requests
Secure file upload	Prevent uploading executable/malicious files	`mimes:` validation restricting allowed file types
`.env` secrets	Never expose credentials	Git-ignored by default, never commit it
API security	Protect endpoints from abuse	Sanctum + rate limiting + validation together

31.1 Rate Limiting Example

```
php
```

```php
Route::middleware('throttle:60,1')->group(function () {
    Route::apiResource('products', ProductController::class);
});
```

Explanation: Limits each user/IP to 60 requests per 1 minute; excess requests get a `429 Too Many Requests` response.

31.2 Common Security Mistakes

- Committing `.env` to Git (leaks database passwords, API keys)
- Using `$request->all()` for mass assignment without `$fillable` set
- Leaving `APP_DEBUG=true` in production (leaks stack traces, file paths, and env details to attackers)
- Storing passwords with `md5()` instead of `Hash::make()`
- Not validating file uploads, allowing any file type through
- Trusting IDs/ownership blindly instead of checking via Policies (e.g. letting any logged-in user edit any order, not just their own)

PART 32 — DATABASE SEEDERS & FACTORIES

32.1 Factories

Simple explanation: A Factory generates realistic fake data for a model — useful for testing and populating a local database quickly.

bash

```bash
php artisan make:factory ProductFactory --model=Product
```

`database/factories/ProductFactory.php`:

php

```php
<?php

namespace Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;

class ProductFactory extends Factory
{
    public function definition(): array
    {
        return [
            'name' => fake()->words(3, true),
            'description' => fake()->sentence(),
            'price' => fake()->randomFloat(2, 10, 1000),
            'stock' => fake()->numberBetween(0, 100),
        ];
    }
}
```

Explanation: `fake()` (Faker library) generates realistic random data — names, sentences, numbers — so you don't write it by hand.

32.2 Seeders

```bash
php artisan make:seeder ProductSeeder
```

`database/seeders/ProductSeeder.php`:

```php
```

```php
<?php

namespace Database\Seeders;

use App\Models\Product;
use Illuminate\Database\Seeder;

class ProductSeeder extends Seeder
{
    public function run(): void
    {
        Product::factory()->count(50)->create();
    }
}
```

`database/seeders/DatabaseSeeder.php`:

```php
public function run(): void
{
    $this->call([
        ProductSeeder::class,
        UserSeeder::class,
    ]);
}
```

32.3 Running Seeders

```bash
php artisan db:seed
php artisan migrate:fresh --seed   # rebuild DB and seed in one command
```

32.4 Factories with Relationships

```php
```

```php
class OrderFactory extends Factory
{
    public function definition(): array
    {
        return [
            'user_id' => \App\Models\User::factory(),
            'total' => fake()->randomFloat(2, 20, 500),
        ];
    }
}
```

```php
Order::factory()
    ->has(OrderItem::factory()->count(3))
    ->create();
```

PART 33 — TESTING

33.1 Why Testing

Simple explanation: Automated tests run your code and verify it behaves correctly, catching bugs before real users do — especially important before deploying changes.

33.2 Unit vs Feature Tests

- **Unit test** — tests a single small piece of logic in isolation (e.g. a calculation function).
- **Feature test** — tests a full flow through the application, like an actual HTTP request hitting a route and checking the response (most common for API testing).

33.3 Pest vs PHPUnit

Laravel currently ships with **Pest** by default in new projects (a simpler, more readable syntax built on top of PHPUnit), though PHPUnit is still fully supported.

33.4 Feature Test — Product CRUD (Pest syntax)

```php
```

```php
<?php

use App\Models\Product;
use App\Models\User;

test('a user can create a product', function () {
    $user = User::factory()->create();

    $response = $this->actingAs($user, 'sanctum')->postJson('/api/products', [
        'name' => 'Laptop',
        'price' => 999.99,
        'stock' => 5,
    ]);

    $response->assertStatus(201);
    $this->assertDatabaseHas('products', ['name' => 'Laptop']);
});

test('creating a product fails without a name', function () {
    $user = User::factory()->create();

    $response = $this->actingAs($user, 'sanctum')->postJson('/api/products', [
        'price' => 999.99,
    ]);

    $response->assertStatus(422)
             ->assertJsonValidationErrors('name');
});

test('guests cannot create a product', function () {
    $response = $this->postJson('/api/products', ['name' => 'Laptop']);
    $response->assertStatus(401);
});
```

33.5 Same Test in PHPUnit Syntax

```
php
```

```php
<?php

namespace Tests\Feature;

use App\Models\User;
use Illuminate\Foundation\Testing\RefreshDatabase;
use Tests\TestCase;

class ProductTest extends TestCase
{
    use RefreshDatabase;

    public function test_a_user_can_create_a_product(): void
    {
        $user = User::factory()->create();

        $response = $this->actingAs($user, 'sanctum')->postJson('/api/products', [
            'name' => 'Laptop',
            'price' => 999.99,
            'stock' => 5,
        ]);

        $response->assertStatus(201);
        $this->assertDatabaseHas('products', ['name' => 'Laptop']);
    }
}
```

Explanation:

- `RefreshDatabase` — resets the test database before each test, so tests don't interfere with each other.
- `actingAs($user, 'sanctum')` — simulates a logged-in user without a real login request.
- `postJson()` — sends a JSON POST request to a route, exactly like React's Axios would.
- `assertStatus()`, `assertDatabaseHas()`, `assertJsonValidationErrors()` — readable assertions checking the outcome.

33.6 Running Tests

```bash
php artisan test
```

PART 34 — ARTISAN COMMAND CHEAT SHEET

Command	What It Does
`php artisan serve`	Starts local dev server
`php artisan route:list`	Lists all registered routes
`php artisan make:model Product -m`	Creates a Model (+migration with `-m`)
`php artisan make:controller ProductController --api`	Creates a REST controller
`php artisan make:migration create_products_table`	Creates a migration file
`php artisan migrate`	Runs pending migrations
`php artisan migrate:rollback`	Undoes the last migration batch
`php artisan migrate:fresh`	Drops all tables and re-migrates
`php artisan migrate:fresh --seed`	Fresh migrate + run seeders
`php artisan make:request StoreProductRequest`	Creates a Form Request validator
`php artisan make:resource ProductResource`	Creates an API Resource
`php artisan make:middleware EnsureUserIsAdmin`	Creates middleware
`php artisan make:policy ProductPolicy --model=Product`	Creates a Policy
`php artisan make:event UserRegistered`	Creates an Event
`php artisan make:listener SendWelcomeEmail --event=UserRegistered`	Creates a Listener
`php artisan make:job SendOrderConfirmationEmail`	Creates a Job
`php artisan make:notification OrderShipped`	Creates a Notification
`php artisan make:seeder ProductSeeder`	Creates a Seeder
`php artisan make:factory ProductFactory`	Creates a Factory
`php artisan db:seed`	Runs seeders

Command	What It Does
`php artisan tinker`	Opens an interactive PHP shell inside your app
`php artisan queue:work`	Starts a queue worker
`php artisan queue:failed`	Lists failed jobs
`php artisan storage:link`	Links public storage folder
`php artisan test`	Runs the test suite
`php artisan config:cache`	Caches config for performance (production)
`php artisan route:cache`	Caches routes for performance (production)
`php artisan optimize:clear`	Clears all Laravel caches
`php artisan key:generate`	Generates a new `APP_KEY`

PART 35 — PROFESSIONAL PROJECT STRUCTURE

```text
```

```
app/
├── Http/
│   ├── Controllers/
│   │   ├── Api/
│   │   │   ├── ProductController.php
│   │   │   ├── OrderController.php
│   │   │   └── AuthController.php
│   │   └── Admin/
│   │       └── DashboardController.php
│   ├── Requests/
│   │   ├── StoreProductRequest.php
│   │   └── UpdateProductRequest.php
│   ├── Resources/
│   │   ├── ProductResource.php
│   │   └── OrderResource.php
│   └── Middleware/
│       └── EnsureUserIsAdmin.php
│
├── Models/
│   ├── User.php
│   ├── Product.php
│   ├── Category.php
│   └── Order.php
│
├── Services/
│   ├── ProductService.php
│   └── OrderService.php
│
├── Repositories/                (only in large projects — Part 25)
│   ├── ProductRepositoryInterface.php
│   └── EloquentProductRepository.php
│
├── Policies/
│   └── ProductPolicy.php
│
├── Events/
│   └── OrderPlaced.php
│
├── Listeners/
│   └── SendOrderConfirmation.php
│
├── Jobs/
│   └── SendOrderConfirmationEmail.php
│
```

```
└── Notifications/
    └── OrderShipped.php
```

Why organize this way: Each folder has one clear responsibility. When a new developer joins the project, they can guess exactly where to find (or add) code just from this structure — this predictability is one of Laravel's biggest strengths for team projects.

PART 36 — GIT & GITHUB

36.1 Basic Commands

bash

```bash
git init                          # start a repo
git status                        # see changed files
git add .                         # stage all changes
git commit -m "Initial commit"    # save a snapshot
git branch feature/auth           # create a branch
git checkout feature/auth         # switch to it
git push origin feature/auth      # push to GitHub
git pull origin main               # fetch + merge latest changes
```

36.2 `.gitignore`

Laravel ships with a sensible default `.gitignore`, but confirm it includes:

text

```
/vendor
/node_modules
.env
/storage/*.key
/public/storage
/storage/logs/*.log
```

36.3 Why `.env` Must NEVER Be Uploaded

Simple explanation: `.env` contains your database password, mail credentials, API keys, and `APP_KEY` (used to encrypt sessions). If pushed to a public GitHub repo, anyone can read your production secrets and potentially access your database or send emails as your app. Instead, commit a `.env.example` with placeholder values, and set real values directly on the server/hosting platform.

PART 37 — DEPLOYMENT

37.1 Deployment Flow

```text
Local Development
       ↓
GitHub (push code)
       ↓
Production Server (pull code)
       ↓
composer install --optimize-autoloader --no-dev
       ↓
Set real .env values
       ↓
php artisan migrate --force
       ↓
php artisan config:cache && route:cache && view:cache
       ↓
Set up queue worker (Supervisor) & storage link
       ↓
Domain + HTTPS configured
       ↓
Live
```

37.2 Hosting Options

Option	Best For
Shared hosting	Very small/simple projects, cheapest, least control
VPS (DigitalOcean, Linode, AWS EC2)	Full control, requires manual server setup
Managed Laravel hosting (Laravel Forge, Laravel Cloud)	Production apps, automated deployment, official support

37.3 Production Environment Variables

```env
APP_ENV=production
APP_DEBUG=false        # CRITICAL — never expose stack traces in production
APP_URL=https://myapp.com
```

37.4 Production Configuration Commands

```bash
composer install --optimize-autoloader --no-dev
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Explanation: These caching commands precompile config/routes/views into optimized files, significantly speeding up production performance (skip them during local development — they make config changes not take effect until you clear the cache).

37.5 Storage & Queue Workers in Production

```bash
php artisan storage:link
```

Queue workers must run continuously — use **Supervisor** (a process manager) to keep `php artisan queue:work` alive and auto-restart it if it crashes.

37.6 Logs & HTTPS

- Check `storage/logs/laravel.log` for production errors.
- Always serve production traffic over **HTTPS** (SSL certificate) — most managed hosts (Forge, Vapor) or Let's Encrypt (via Certbot on a VPS) handle this for you.
- Point your domain's DNS to your server's IP address, and configure it in your web server (Nginx/Apache).

PART 38 — FINAL REAL-WORLD PROJECT: E-COMMERCE MANAGEMENT SYSTEM

Stack: Laravel REST API + React frontend + Oracle (or MySQL) database.

38.1 Final Project Structure

```text
```

```
app/
 ├── Http/
 │   ├── Controllers/Api/
 │   │    ├── AuthController.php
 │   │    ├── CategoryController.php
 │   │    ├── ProductController.php
 │   │    ├── CartController.php
 │   │    ├── OrderController.php
 │   │    └── Admin/DashboardController.php
 │   ├── Requests/
 │   │    ├── StoreProductRequest.php
 │   │    ├── UpdateProductRequest.php
 │   │    └── StoreCategoryRequest.php
 │   ├── Resources/
 │   │    ├── ProductResource.php
 │   │    ├── CategoryResource.php
 │   │    ├── CartResource.php
 │   │    └── OrderResource.php
 │   └── Middleware/
 │        └── EnsureUserIsAdmin.php
 ├── Models/
 │    ├── User.php
 │    ├── Category.php
 │    ├── Product.php
 │    ├── Cart.php
 │    ├── CartItem.php
 │    ├── Order.php
 │    └── OrderItem.php
 ├── Services/
 │    ├── CartService.php
 │    └── OrderService.php
 └── Policies/
      └── ProductPolicy.php

database/migrations/
 ├── create_users_table.php (built-in, modified with role)
 ├── create_categories_table.php
 ├── create_products_table.php
 ├── create_carts_table.php
 ├── create_cart_items_table.php
 ├── create_orders_table.php
 └── create_order_items_table.php

routes/api.php
```

38.2 Migrations

Users (add `role` to the default migration):

```php
Schema::table('users', function (Blueprint $table) {
    $table->string('role')->default('customer')->after('email'); // admin | customer
});
```

Categories:

```php
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('slug')->unique();
    $table->timestamps();
});
```

Products:

```php
Schema::create('products', function (Blueprint $table) {
    $table->id();
    $table->foreignId('category_id')->constrained()->onDelete('cascade');
    $table->string('name');
    $table->text('description')->nullable();
    $table->decimal('price', 10, 2);
    $table->integer('stock')->default(0);
    $table->string('image')->nullable();
    $table->timestamps();
});
```

Carts & Cart Items:

```php
```

```php
Schema::create('carts', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->unique()->constrained()->onDelete('cascade');
    $table->timestamps();
});

Schema::create('cart_items', function (Blueprint $table) {
    $table->id();
    $table->foreignId('cart_id')->constrained()->onDelete('cascade');
    $table->foreignId('product_id')->constrained()->onDelete('cascade');
    $table->integer('quantity')->default(1);
    $table->timestamps();
});
```

Orders & Order Items:

```php
Schema::create('orders', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->decimal('total', 10, 2);
    $table->string('status')->default('pending'); // pending | processing | shipped |
    $table->timestamps();
});

Schema::create('order_items', function (Blueprint $table) {
    $table->id();
    $table->foreignId('order_id')->constrained()->onDelete('cascade');
    $table->foreignId('product_id')->constrained();
    $table->integer('quantity');
    $table->decimal('price', 10, 2); // price at time of purchase
    $table->timestamps();
});
```

38.3 Models

`app/Models/User.php` (key additions):

```php
```

```php
<?php

namespace App\Models;

use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;

    protected $fillable = ['name', 'email', 'password', 'role'];
    protected $hidden = ['password', 'remember_token'];

    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }

    public function orders()
    {
        return $this->hasMany(Order::class);
    }

    public function cart()
    {
        return $this->hasOne(Cart::class);
    }
}
```

`app/Models/Category.php`:

```php
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    protected $fillable = ['name', 'slug'];

    public function products()
    {
        return $this->hasMany(Product::class);
    }
}
```

`app/Models/Product.php`:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Product extends Model
{
    protected $fillable = ['category_id', 'name', 'description', 'price', 'stock', 'ima

    public function category()
    {
        return $this->belongsTo(Category::class);
    }
}
```

`app/Models/Cart.php` & `CartItem.php`:

```php
```

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Cart extends Model
{
    protected $fillable = ['user_id'];

    public function items()
    {
        return $this->hasMany(CartItem::class);
    }
}
```

php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class CartItem extends Model
{
    protected $fillable = ['cart_id', 'product_id', 'quantity'];

    public function product()
    {
        return $this->belongsTo(Product::class);
    }
}
```

`app/Models/Order.php` & `OrderItem.php`:

php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    protected $fillable = ['user_id', 'total', 'status'];

    public function items()
    {
        return $this->hasMany(OrderItem::class);
    }

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class OrderItem extends Model
{
    protected $fillable = ['order_id', 'product_id', 'quantity', 'price'];

    public function product()
    {
        return $this->belongsTo(Product::class);
    }
}
```

38.4 Form Requests

`app/Http/Requests/StoreProductRequest.php`:

php

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreProductRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->isAdmin();
    }

    public function rules(): array
    {
        return [
            'category_id' => 'required|exists:categories,id',
            'name' => 'required|string|max:255',
            'description' => 'nullable|string',
            'price' => 'required|numeric|min:0',
            'stock' => 'required|integer|min:0',
            'image' => 'nullable|image|mimes:jpg,jpeg,png,webp|max:2048',
        ];
    }
}
```

`app/Http/Requests/StoreCategoryRequest.php`:

php

```php
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreCategoryRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->isAdmin();
    }

    public function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'slug' => 'required|string|unique:categories,slug',
        ];
    }
}
```

38.5 API Resources

`app/Http/Resources/ProductResource.php`:

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class ProductResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'description' => $this->description,
            'price' => (float) $this->price,
            'stock' => $this->stock,
            'in_stock' => $this->stock > 0,
            'image_url' => $this->image ? asset('storage/' . $this->image) : null,
            'category' => new CategoryResource($this->whenLoaded('category')),
            'created_at' => $this->created_at,
        ];
    }
}
```

`app/Http/Resources/CategoryResource.php`:

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class CategoryResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'name' => $this->name,
            'slug' => $this->slug,
        ];
    }
}
```

`app/Http/Resources/OrderResource.php`:

```php
```

```php
<?php

namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class OrderResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'status' => $this->status,
            'total' => (float) $this->total,
            'items' => $this->items->map(fn($item) => [
                'product' => $item->product->name,
                'quantity' => $item->quantity,
                'price' => (float) $item->price,
            ]),
            'created_at' => $this->created_at,
        ];
    }
}
```

38.6 Middleware

`app/Http/Middleware/EnsureUserIsAdmin.php`:

```
php
```

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class EnsureUserIsAdmin
{
    public function handle(Request $request, Closure $next)
    {
        if (!$request->user() || !$request->user()->isAdmin()) {
            return response()->json(['message' => 'Forbidden — admin access required'],
        }

        return $next($request);
    }
}
```

Register alias in `bootstrap/app.php`:

```php
->withMiddleware(function (Middleware $middleware) {
    $middleware->alias(['admin' => \App\Http\Middleware\EnsureUserIsAdmin::class]);
})
```

38.7 Services

`app/Services/CartService.php`:

```php
```

```php
<?php

namespace App\Services;

use App\Models\Cart;
use App\Models\Product;
use App\Models\User;

class CartService
{
    public function getOrCreateCart(User $user): Cart
    {
        return Cart::firstOrCreate(['user_id' => $user->id]);
    }

    public function addItem(User $user, int $productId, int $quantity): Cart
    {
        $cart = $this->getOrCreateCart($user);
        $product = Product::findOrFail($productId);

        $item = $cart->items()->where('product_id', $product->id)->first();

        if ($item) {
            $item->update(['quantity' => $item->quantity + $quantity]);
        } else {
            $cart->items()->create(['product_id' => $product->id, 'quantity' => $quant
        }

        return $cart->load('items.product');
    }

    public function updateQuantity(User $user, int $itemId, int $quantity): Cart
    {
        $cart = $this->getOrCreateCart($user);
        $item = $cart->items()->findOrFail($itemId);
        $item->update(['quantity' => $quantity]);

        return $cart->load('items.product');
    }

    public function removeItem(User $user, int $itemId): Cart
    {
        $cart = $this->getOrCreateCart($user);
        $cart->items()->where('id', $itemId)->delete();
```

```
        return $cart->load('items.product');
    }
}
```

`app/Services/OrderService.php`:

```php
```

```php
<?php

namespace App\Services;

use App\Models\Order;
use App\Models\User;
use Illuminate\Support\Facades\DB;

class OrderService
{
    public function __construct(protected CartService $cartService) {}

    public function createFromCart(User $user): Order
    {
        $cart = $this->cartService->getOrCreateCart($user)->load('items.product');

        if ($cart->items->isEmpty()) {
            throw new \Exception('Cart is empty');
        }

        return DB::transaction(function () use ($cart, $user) {
            $total = $cart->items->sum(fn($item) => $item->product->price * $item->quar

            $order = Order::create([
                'user_id' => $user->id,
                'total' => $total,
                'status' => 'pending',
            ]);

            foreach ($cart->items as $item) {
                $order->items()->create([
                    'product_id' => $item->product_id,
                    'quantity' => $item->quantity,
                    'price' => $item->product->price,
                ]);

                $item->product->decrement('stock', $item->quantity);
            }

            $cart->items()->delete(); // empty the cart after order creation

            return $order->load('items.product');
        });
```

```
    }
}
```

Explanation: `DB::transaction()` wraps everything in a database transaction — if any step fails (e.g. an exception mid-loop), **all** changes roll back automatically, so you never end up with a half-created order or incorrectly reduced stock.

38.8 Controllers

`app/Http/Controllers/Api/AuthController.php` — see Part 18.2 (used as-is, with `role` defaulting to `customer` on register).

`app/Http/Controllers/Api/CategoryController.php`:

```php
```

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Requests\StoreCategoryRequest;
use App\Http\Resources\CategoryResource;
use App\Models\Category;

class CategoryController extends Controller
{
    public function index()
    {
        return CategoryResource::collection(Category::all());
    }

    public function store(StoreCategoryRequest $request)
    {
        $category = Category::create($request->validated());
        return new CategoryResource($category);
    }

    public function update(StoreCategoryRequest $request, Category $category)
    {
        $category->update($request->validated());
        return new CategoryResource($category);
    }

    public function destroy(Category $category)
    {
        $category->delete();
        return response()->json(['message' => 'Category deleted']);
    }
}
```

`app/Http/Controllers/Api/ProductController.php`:

```php
```

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Requests\StoreProductRequest;
use App\Http\Resources\ProductResource;
use App\Models\Product;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    public function index(Request $request)
    {
        $query = Product::with('category');

        if ($request->filled('search')) {
            $query->where('name', 'like', '%' . $request->search . '%');
        }
        if ($request->filled('category')) {
            $query->whereHas('category', fn($q) => $q->where('slug', $request->category
        }
        if ($request->filled('min_price')) {
            $query->where('price', '>=', $request->min_price);
        }
        if ($request->filled('max_price')) {
            $query->where('price', '<=', $request->max_price);
        }

        $query->orderBy($request->get('sort', 'created_at'), $request->get('order', 'd

        return ProductResource::collection($query->paginate($request->get('per_page',
    }

    public function store(StoreProductRequest $request)
    {
        $data = $request->validated();

        if ($request->hasFile('image')) {
            $data['image'] = $request->file('image')->store('products', 'public');
        }

        $product = Product::create($data);
        return new ProductResource($product);
```

```
    }

    public function show(Product $product)
    {
        return new ProductResource($product->load('category'));
    }

    public function update(StoreProductRequest $request, Product $product)
    {
        $data = $request->validated();

        if ($request->hasFile('image')) {
            $data['image'] = $request->file('image')->store('products', 'public');
        }

        $product->update($data);
        return new ProductResource($product);
    }

    public function destroy(Product $product)
    {
        $product->delete();
        return response()->json(['message' => 'Product deleted']);
    }
}
```

`app/Http/Controllers/Api/CartController.php`:

```
php
```

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Services\CartService;
use Illuminate\Http\Request;

class CartController extends Controller
{
    public function __construct(protected CartService $cartService) {}

    public function index(Request $request)
    {
        $cart = $this->cartService->getOrCreateCart($request->user())->load('items.pro
        return response()->json($cart);
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'product_id' => 'required|exists:products,id',
            'quantity' => 'required|integer|min:1',
        ]);

        $cart = $this->cartService->addItem($request->user(), $validated['product_id']
        return response()->json($cart);
    }

    public function update(Request $request, int $itemId)
    {
        $validated = $request->validate(['quantity' => 'required|integer|min:1']);
        $cart = $this->cartService->updateQuantity($request->user(), $itemId, $validat
        return response()->json($cart);
    }

    public function destroy(Request $request, int $itemId)
    {
        $cart = $this->cartService->removeItem($request->user(), $itemId);
        return response()->json($cart);
    }
}
```

`app/Http/Controllers/Api/OrderController.php`:

```php
```

```php
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Http\Resources\OrderResource;
use App\Models\Order;
use App\Services\OrderService;
use Illuminate\Http\Request;

class OrderController extends Controller
{
    public function __construct(protected OrderService $orderService) {}

    public function store(Request $request)
    {
        $order = $this->orderService->createFromCart($request->user());
        return new OrderResource($order);
    }

    public function index(Request $request)
    {
        $orders = $request->user()->orders()->with('items.product')->latest()->get();
        return OrderResource::collection($orders);
    }

    public function show(Request $request, Order $order)
    {
        abort_if($order->user_id !== $request->user()->id && !$request->user()->isAdmi
        return new OrderResource($order->load('items.product'));
    }

    // Admin only
    public function adminIndex()
    {
        $orders = Order::with('user', 'items.product')->latest()->get();
        return OrderResource::collection($orders);
    }

    public function updateStatus(Request $request, Order $order)
    {
        $validated = $request->validate([
            'status' => 'required|in:pending,processing,shipped,delivered,cancelled',
        ]);
```

```
        $order->update($validated);
        return new OrderResource($order);
    }
}
```

`app/Http/Controllers/Api/Admin/DashboardController.php`:

```php
<?php

namespace App\Http\Controllers\Api\Admin;

use App\Http\Controllers\Controller;
use App\Models\Order;
use App\Models\Product;
use App\Models\User;

class DashboardController extends Controller
{
    public function index()
    {
        return response()->json([
            'total_users' => User::where('role', 'customer')->count(),
            'total_products' => Product::count(),
            'total_orders' => Order::count(),
            'total_revenue' => Order::where('status', '!=', 'cancelled')->sum('total'),
            'pending_orders' => Order::where('status', 'pending')->count(),
        ]);
    }
}
```

38.9 Routes

`routes/api.php`:

```php
```

```php
<?php

use App\Http\Controllers\Api\Admin\DashboardController;
use App\Http\Controllers\Api\AuthController;
use App\Http\Controllers\Api\CartController;
use App\Http\Controllers\Api\CategoryController;
use App\Http\Controllers\Api\OrderController;
use App\Http\Controllers\Api\ProductController;
use Illuminate\Support\Facades\Route;

// Public routes
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);
Route::get('/products', [ProductController::class, 'index']);
Route::get('/products/{product}', [ProductController::class, 'show']);
Route::get('/categories', [CategoryController::class, 'index']);

// Authenticated (any logged-in user)
Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/user', [AuthController::class, 'user']);

    Route::get('/cart', [CartController::class, 'index']);
    Route::post('/cart', [CartController::class, 'store']);
    Route::put('/cart/{itemId}', [CartController::class, 'update']);
    Route::delete('/cart/{itemId}', [CartController::class, 'destroy']);

    Route::post('/orders', [OrderController::class, 'store']);
    Route::get('/orders', [OrderController::class, 'index']);
    Route::get('/orders/{order}', [OrderController::class, 'show']);
});

// Admin only
Route::middleware(['auth:sanctum', 'admin'])->prefix('admin')->group(function () {
    Route::post('/categories', [CategoryController::class, 'store']);
    Route::put('/categories/{category}', [CategoryController::class, 'update']);
    Route::delete('/categories/{category}', [CategoryController::class, 'destroy']);

    Route::post('/products', [ProductController::class, 'store']);
    Route::put('/products/{product}', [ProductController::class, 'update']);
    Route::delete('/products/{product}', [ProductController::class, 'destroy']);

    Route::get('/orders', [OrderController::class, 'adminIndex']);
    Route::put('/orders/{order}/status', [OrderController::class, 'updateStatus']);
```

```
    Route::get('/dashboard', [DashboardController::class, 'index']);
});
```

38.10 Database Configuration (MySQL and Oracle side-by-side)

env

```
# For MySQL
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=ecommerce
DB_USERNAME=root
DB_PASSWORD=

# For Oracle (swap the block above with this one)
# DB_CONNECTION=oracle
# DB_HOST=127.0.0.1
# DB_PORT=1521
# DB_DATABASE=XE
# DB_SERVICE_NAME=XE
# DB_USERNAME=system
# DB_PASSWORD=oracle
```

Every model, controller, and service above works unchanged on either database — that's the value of Eloquent.

PART 39 — API DOCUMENTATION

Method	Endpoint	Purpose	Auth
POST	/api/register	Register a new user	No
POST	/api/login	Login and receive token	No
POST	/api/logout	Logout (revoke token)	Yes
GET	/api/user	Get current logged-in user	Yes
GET	/api/products	List/search/filter products	No
GET	/api/products/{id}	Get single product	No
POST	/api/admin/products	Create product	Admin
PUT	/api/admin/products/{id}	Update product	Admin
DELETE	/api/admin/products/{id}	Delete product	Admin
GET	/api/categories	List categories	No
POST	/api/admin/categories	Create category	Admin
PUT	/api/admin/categories/{id}	Update category	Admin
DELETE	/api/admin/categories/{id}	Delete category	Admin
GET	/api/cart	View current user's cart	Yes
POST	/api/cart	Add item to cart	Yes
PUT	/api/cart/{itemId}	Update cart item quantity	Yes
DELETE	/api/cart/{itemId}	Remove cart item	Yes
POST	/api/orders	Create order from cart (checkout)	Yes
GET	/api/orders	List current user's orders	Yes
GET	/api/orders/{id}	View a single order	Yes
GET	/api/admin/orders	List all orders	Admin
PUT	/api/admin/orders/{id}/status	Update order status	Admin
GET	/api/admin/dashboard	Store statistics	Admin

PART 40 — LARAVEL INTERVIEW PREPARATION

Beginner Questions (30)

1. What is Laravel? Simple Answer: A PHP framework for building web apps and APIs with built-in tools for routing, database access, auth, and more. Real-world Example: Used to build a REST API backend for a React e-commerce store.

2. What is MVC? Simple Answer: Model-View-Controller — a pattern separating data (Model), display (View), and logic (Controller). Real-world Example: `Product` model, JSON response as "view," `ProductController` handling requests.

3. What is Composer? Simple Answer: PHP's package manager, like npm for Node.js. Real-world Example: `composer require laravel/sanctum` installs the Sanctum auth package.

4. What is Artisan? Simple Answer: Laravel's command-line tool for generating code and running tasks. Real-world Example: `php artisan make:model Product -m` creates a model and migration.

5. What is a route? Simple Answer: A rule connecting a URL to code that should run. Real-world Example: `Route::get('/products', [ProductController::class, 'index'])`.

6. What is a Controller? Simple Answer: A class that groups request-handling logic for related routes. Real-world Example: `ProductController` handles all product-related requests.

7. What is a Model? Simple Answer: A PHP class representing one database table. Real-world Example: `Product` model represents the `products` table.

8. What is Eloquent? Simple Answer: Laravel's built-in ORM for working with the database using PHP objects instead of raw SQL. Real-world Example: `Product::where('price', '>', 100)->get()`.

9. What is a migration? Simple Answer: A version-controlled file describing a change to the database structure. Real-world Example: `create_products_table` migration creates the products table.

10. What is `$fillable`? Simple Answer: A whitelist of fields allowed to be mass-assigned on a model. Real-world Example: `protected $fillable = ['name', 'price'];` prevents unexpected fields from being saved.

11. What is middleware? Simple Answer: Code that runs before/after a request reaches the controller, often to check something. Real-world Example: `auth:sanctum` middleware blocks unauthenticated requests.

12. What is the `.env` file? Simple Answer: A file storing environment-specific configuration and secrets. Real-world Example: Database credentials, mail settings, `APP_KEY`.

13. What is `APP_KEY` used for? Simple Answer: A secret key Laravel uses to encrypt sessions and cookies. Real-world Example: Generated with `php artisan key:generate`.

14. What's the difference between `get()` and `find()`? Simple Answer: `get()` returns all matching rows as a collection; `find()` returns a single row by primary key. Real-world Example: `Product::find(5)` gets product with ID 5.

15. What is validation used for? Simple Answer: Ensuring incoming request data meets required rules before processing it. Real-world Example: `'email' => 'required|email'` rejects malformed emails.

16. What is a Form Request? Simple Answer: A dedicated class holding validation rules, keeping controllers clean. Real-world Example: `StoreProductRequest` validates product creation data.

17. What is Blade? Simple Answer: Laravel's templating engine for generating HTML. Real-world Example: `{{ $product->name }}` outputs a variable safely.

18. What does `php artisan serve` do? Simple Answer: Starts a local development server. Real-world Example: Runs the app at `http://127.0.0.1:8000` for local testing.

19. What is a primary key? Simple Answer: A unique identifier for each row in a table. Real-world Example: The `id` column on the `products` table.

20. What is a foreign key? Simple Answer: A column pointing to another table's primary key, creating a relationship. Real-world Example: `products.category_id` points to `categories.id`.

21. What is `hasMany`? Simple Answer: An Eloquent relationship meaning "this model owns many of another model." Real-world Example: `User::hasMany(Order::class)` — a user has many orders.

22. What is `belongsTo`? Simple Answer: The inverse relationship — "this model belongs to one of another model." Real-world Example: `Order::belongsTo(User::class)`.

23. What is JSON, and why does an API return it? Simple Answer: A lightweight text format for structured data, easily read by JavaScript. Real-world Example: `{"id": 1, "name": "Laptop"}` returned to a React app.

24. What is CRUD? Simple Answer: Create, Read, Update, Delete — the four basic data operations. Real-world Example: A Product API's `store`, `index`/`show`, `update`, `destroy` methods.

25. What is the difference between `POST` and `GET`? Simple Answer: `GET` retrieves data without side effects; `POST` sends data to create/change something. Real-world Example: `GET /products` lists products; `POST /products` creates one.

26. What is `php artisan migrate` for? Simple Answer: Runs all pending migrations, applying them to the database. Real-world Example: Creates the `products` table for the first time.

27. What is a seeder? Simple Answer: A script that inserts sample/test data into the database. Real-world Example: `ProductSeeder` inserts 50 fake products for testing.

28. What does `dd()` do? Simple Answer: "Dump and die" — prints a variable and stops script execution, used for debugging. Real-world Example: `dd($request->all());` to inspect incoming request data.

29. What is the difference between `include` and Laravel's autoloading? Simple Answer: `include` manually pulls in a file; Laravel autoloads classes automatically via Composer, so you rarely use `include`/`require` yourself. Real-world Example: `use App\Models\Product;` — no manual `require` needed.

30. What is a namespace? Simple Answer: An organizational "address" for a class, preventing name collisions. Real-world Example: `namespace App\Models;` matches the folder `app/Models/`.

Intermediate Questions (30)

31. What is dependency injection? Simple Answer: Automatically providing a class the objects it needs, instead of it creating them manually. Real-world Example: `public function __construct(ProductService $service)` — Laravel supplies `$service` automatically.

32. What is the difference between `hasOne` and `belongsTo`? Simple Answer: `hasOne` is on the "owning" side (has the other model's foreign key pointing to it); `belongsTo` is on the side holding the foreign key. Real-world Example: `User::hasOne(Profile::class)`, `Profile::belongsTo(User::class)`.

33. What is `belongsToMany`? Simple Answer: A many-to-many relationship using a pivot table. Real-world Example: `Student::belongsToMany(Course::class)` via a `course_student` pivot table.

34. What is eager loading and why does it matter? Simple Answer: Loading related data upfront with `with()` to avoid the N+1 query problem. Real-world Example: `Product::with('category')->get()` avoids one extra query per product.

35. What is the N+1 problem? Simple Answer: Running 1 query for a list plus N extra queries for each item's relationship, causing performance issues. Real-world Example: Looping over 100 products and calling `->category->name` without eager loading triggers 101 queries.

36. What is an API Resource? Simple Answer: A class transforming a model into a controlled JSON shape before sending it to the frontend. Real-world Example: `ProductResource` hides internal fields and formats the price as a float.

37. What is Sanctum? Simple Answer: Laravel's lightweight package for API token authentication, ideal for SPAs. Real-world Example: React stores a Sanctum token and sends it as a Bearer header.

38. What is the difference between authentication and authorization? Simple Answer: Authentication confirms identity ("who are you"); authorization confirms permission ("what can you do"). Real-world Example: Login = authentication; only admins can delete products = authorization.

39. What is a Policy? Simple Answer: A class holding authorization rules for a specific model. Real-world Example: `ProductPolicy::update()` checks if a user can edit a specific product.

40. What is a Gate? Simple Answer: A simple closure-based authorization rule, not tied to one specific model. Real-world Example: `Gate::define('access-admin-panel', ...)`.

41. What is mass assignment, and why is it dangerous unprotected? Simple Answer: Creating/updating a model from an entire array at once; without `$fillable`, an attacker could set unintended fields. Real-world Example: Sending `is_admin: true` in a registration request if not protected.

42. What is pagination and how does Laravel handle it? Simple Answer: Splitting large result sets into pages; Laravel's `paginate()` method handles the logic and metadata automatically. Real-world Example: `Product::paginate(10)` returns 10 products plus page metadata.

43. What is a Service class? Simple Answer: A plain class holding business logic, keeping controllers thin. Real-world Example: `OrderService::createFromCart()` handles checkout logic.

44. What is CORS and why do React + Laravel projects need it configured? Simple Answer: A browser rule blocking cross-origin requests unless explicitly allowed; needed because React and Laravel typically run on different ports/domains. Real-world Example: `config/cors.php` allowing `http://localhost:3000`.

45. What is a queue? Simple Answer: A system for running slow tasks in the background instead of during the request. Real-world Example: Sending a confirmation email via a queued job after checkout.

46. What is a Job? Simple Answer: A class representing one unit of background work, dispatched to a queue. Real-world Example: `SendOrderConfirmationEmail::dispatch($order)`.

47. What is an Event/Listener pair used for? Simple Answer: Decoupling "something happened" from "what should happen in response." Real-world Example: `UserRegistered` event triggers a `SendWelcomeEmail` listener.

48. What is route model binding? Simple Answer: Laravel automatically resolving a model instance from a route parameter. Real-world Example: `Route::get('/products/{product}', ...)` injects the actual `Product` model, not just an ID.

49. What is the difference between `PUT` and `PATCH`? Simple Answer: `PUT` conventionally replaces an entire resource; `PATCH` updates part of it. In Laravel practice both often map to the same `update()` method. Real-world Example: Updating just a product's price could use `PATCH` semantically.

50. What is a Form Request's `authorize()` method for? Simple Answer: Determines whether the current user is allowed to make this specific request, before validation even runs. Real-world Example: `return $this->user()->isAdmin();` in `StoreProductRequest`.

51. What is the purpose of database transactions? Simple Answer: Grouping multiple database operations so they all succeed together or all roll back together. Real-world Example: `DB::transaction()` ensures an order and its items are never partially created.

52. What is soft deleting? Simple Answer: Marking a row as deleted (via a `deleted_at` timestamp) instead of permanently removing it. Real-world Example: A soft-deleted product can be restored from a "trash" view.

53. What is `whereHas()` used for? Simple Answer: Filtering a model based on a condition on its related model. Real-world Example: `Product::whereHas('category', fn($q) => $q->where('slug', 'electronics'))`.

54. What is a factory? Simple Answer: A class generating realistic fake data for a model, used in seeding and testing. Real-world Example: `ProductFactory` generates 50 fake products.

55. What is the difference between a Unit test and a Feature test? Simple Answer: Unit tests check one isolated piece of logic; Feature tests check a full flow, like an HTTP request end-to-end. Real-world Example: A Feature test posting to `/api/products` and asserting a 201 response.

56. What is rate limiting? Simple Answer: Restricting how many requests a client can make in a given time window. Real-world Example: `throttle:60,1` middleware allows 60 requests per minute.

57. What is the difference between `$request->all()` and `$request->validated()`? Simple Answer: `all()` returns every submitted field unfiltered; `validated()` returns only the fields that passed validation rules. Real-world Example: Using `validated()` prevents unexpected extra fields from being mass-assigned.

58. What is a Repository, and when should you use one? Simple Answer: A class isolating database query logic behind an interface; best reserved for large projects needing data-source flexibility. Real-world Example: `ProductRepositoryInterface` swappable between Eloquent and another data source.

59. What is `storage:link` for? Simple Answer: Creates a symbolic link making uploaded files in `storage/app/public` accessible via a public URL. Real-world Example: Product images become reachable at `/storage/products/image.jpg`.

60. What is the purpose of API versioning (e.g. `/api/v1/`)? Simple Answer: Allows changing/improving an API without breaking existing clients still using an older version. Real-world Example: Releasing `/api/v2/products` with a new response shape while `/api/v1/products` still works for older app versions.

Advanced Questions (30)

61. How does Laravel's Service Container work? Simple Answer: It's a system that manages class creation and automatically resolves dependencies (dependency injection) throughout the app. Real-world Example: Type-hinting `ProductService` in a controller constructor gets it built automatically.

62. What is the difference between binding a class and binding an interface in the Service Container? Simple Answer: Binding an interface to a concrete class lets you swap implementations without changing code that depends on the interface. Real-world Example: `$this->app->bind(ProductRepositoryInterface::class, EloquentProductRepository::class)`.

63. What are Service Providers? Simple Answer: Classes where you register bindings, event listeners, and bootstrap application services — the central place Laravel "wires everything up." Real-world Example: `AppServiceProvider::register()` binding a repository interface.

64. Explain the full HTTP request lifecycle in Laravel. Simple Answer: Request enters via `public/index.php` → kernel bootstraps → global middleware → router matches → route middleware → controller → response → middleware again → sent back. Real-world Example: An admin-only route runs `auth` then `admin` middleware before reaching the controller.

65. What is the difference between `Route::resource` and `Route::apiResource`? Simple Answer: `resource` generates all 7 RESTful routes (including `create`/`edit` for Blade forms); `apiResource` excludes those two, since APIs don't need HTML forms. Real-world Example: `Route::apiResource('products', ProductController::class)`.

66. How do database indexes improve performance, and what's the trade-off? Simple Answer: Indexes speed up lookups on indexed columns but slightly slow down writes (inserts/updates) since the index must also update. Real-world Example: Indexing `products.sku` speeds up search-by-SKU queries.

67. What is a pivot table, and how do you add extra columns to it? Simple Answer: A middle table connecting two models in a many-to-many relationship; extra columns are accessed via `withPivot()`. Real-world Example: `belongsToMany(Course::class)->withPivot('enrolled_at')`.

68. What is `whereHas` vs `with` — when do you use each? Simple Answer: `with()` eager-loads related data for display; `whereHas()` filters the main query based on a related model's condition. Real-world Example: `Product::with('category')->whereHas('category', fn($q) => $q->where('active', true))->get()`.

69. How does Laravel handle N+1 query prevention besides `with()`? Simple Answer: `load()` for lazy eager loading after the fact, and tools like Laravel Debugbar/Telescope to detect N+1 issues during development. Real-world Example: `$products->load('category')` after already fetching `$products`.

70. What is the Repository Pattern's main trade-off? Simple Answer: Adds an abstraction layer that can improve testability/flexibility but adds boilerplate for simple CRUD apps where Eloquent already suffices. Real-world Example: Overkill for a small blog; useful for a large system swapping data sources.

71. How do queues improve API response times, and what infrastructure do they require? Simple Answer: Slow tasks are deferred to background workers instead of blocking the request; requires a queue driver (database/Redis) and a running worker process. Real-world Example: Order confirmation emails dispatched as a Job, processed by `php artisan queue:work`.

72. What happens if a queued Job fails repeatedly? Simple Answer: After exceeding `$tries`, it's moved to the `failed_jobs` table for inspection and manual/automatic retry. Real-world Example: `php artisan queue:retry all` reprocesses every failed job.

73. How do Laravel Events differ from directly calling a method? Simple Answer: Events decouple the trigger from the reaction — multiple listeners can react independently, and new listeners can be added without touching the original code. Real-world Example: Adding a `LogUserRegistration` listener later without modifying the registration controller.

74. What is the difference between `hasMany` and `hasManyThrough`? Simple Answer: `hasManyThrough` accesses a distant relationship through an intermediate model, in one step. Real-world Example: `Country::hasManyThrough(Post::class, User::class)` — posts belonging to users belonging to a country.

75. How does Laravel's exception handling work in Laravel 11+? Simple Answer: Centralized in `bootstrap/app.php` via `->withExceptions()`, where you register custom rendering logic per exception type. Real-world Example: Rendering `ModelNotFoundException` as a clean JSON 404 for API routes.

76. What is database transaction isolation and why does it matter for checkout flows? Simple Answer: Ensures concurrent operations on the same data don't corrupt each other; critical when multiple users might buy the last unit of stock simultaneously. Real-world Example: Wrapping stock decrement + order creation in `DB::transaction()`.

77. How would you prevent overselling stock in a high-traffic checkout system? Simple Answer: Use database-level locking (`lockForUpdate()`) inside a transaction to prevent race

conditions on stock quantity. Real-world Example: `Product::where('id', $id)->lockForUpdate()->first()` before decrementing stock.

78. What is the difference between `sync()`, `attach()`, and `detach()` on a many-to-many relationship? Simple Answer: `attach()` adds a relation, `detach()` removes one, `sync()` sets the exact final list, removing anything not included. Real-world Example: `$student->courses()->sync([1, 2, 3])` enrolls in exactly those three courses.

79. How does Laravel's cache system work, and where would you use it? Simple Answer: Stores expensive-to-compute results (config, query results) in fast storage (file, Redis) to avoid recomputation. Real-world Example: Caching a homepage's featured-products list for 10 minutes.

80. What are Laravel Policies vs Middleware — when do you choose which? Simple Answer: Middleware is for broad, route-level checks (e.g. "is admin"); Policies are for fine-grained, per-model, per-action checks (e.g. "can this user edit THIS order"). Real-world Example: `admin` middleware blocks the whole admin route group; `OrderPolicy::view()` checks if a customer owns that specific order.

81. How do Form Requests improve testability compared to inline validation? Simple Answer: Validation rules live in a dedicated, reusable, independently testable class rather than being buried inside a controller method. Real-world Example: Testing `StoreProductRequest::rules()` directly without hitting an HTTP endpoint.

82. What's the purpose of API Resource collections vs single resources? Simple Answer: `Resource::collection()` wraps a list with consistent formatting (and pagination metadata when applicable); a single `new Resource()` wraps one item. Real-world Example: `ProductResource::collection($products)` for `index`, `new ProductResource($product)` for `show`.

83. How does Sanctum differ from Passport? Simple Answer: Sanctum is lightweight, ideal for SPAs and simple token auth; Passport implements full OAuth2, suited for third-party API access delegation. Real-world Example: A React SPA uses Sanctum; a public API allowing third-party apps to request access on a user's behalf uses Passport.

84. What is idempotency, and why does it matter for payment/order endpoints? Simple Answer: An idempotent operation produces the same result no matter how many times it's repeated — important so a duplicated network request (e.g. double-click) doesn't create two orders/charges. Real-world Example: Using a unique idempotency key on checkout requests to prevent duplicate orders.

85. How would you structure a Laravel app for multi-tenancy (multiple companies sharing one app)? Simple Answer: Either separate databases per tenant, or a shared database with a `tenant_id` column and global query scopes filtering by tenant automatically. Real-world Example: A global scope automatically adding `where('tenant_id', currentTenant())` to every query.

86. What is a global scope in Eloquent? Simple Answer: A constraint automatically applied to every query for a model, without repeating it manually. Real-world Example: Automatically excluding soft-deleted rows (built into Laravel's `SoftDeletes` trait).

87. How does Laravel handle database connection pooling and performance at scale? Simple Answer: Laravel itself doesn't pool connections (PHP is typically stateless per-request); performance at scale relies on efficient queries, caching, queues, and infrastructure like read replicas or connection poolers (e.g. PgBouncer) configured outside Laravel. Real-world Example: Using a read-replica database connection for heavy reporting queries.

88. What's the benefit of using API Resources over `$product->toArray()` directly? Simple Answer: Resources give explicit, controlled, and evolvable output shape rather than exposing every raw database column automatically. Real-world Example: Hiding an internal `cost_price` field that shouldn't reach the frontend.

89. How do you handle versioned or backward-incompatible database schema changes safely in production? Simple Answer: Use additive migrations (add new column, backfill, then remove old column in a later deploy) rather than destructive changes in one step, to avoid downtime or data loss. Real-world Example: Renaming a column across two deploys: add new column → dual-write → migrate data → remove old column.

90. What are the trade-offs of the Repository + Service layered architecture vs a "fat model, thin controller" approach? Simple Answer: Layered architecture improves testability and separation of concerns for large teams/projects, at the cost of more files and indirection; "fat model" is faster to build for small apps but can get messy as complexity grows. Real-world Example: A large e-commerce platform benefits from Services/Repositories; a small internal tool doesn't need them.

PART 41 — COMMON ERRORS & TROUBLESHOOTING

"Class not found"

1. Why it happens: Autoloader doesn't know about a new/moved class, or a namespace typo.
2. How to check: Look at the `namespace` line vs the file's actual folder path.
3. How to fix: Run `composer dump-autoload`; make sure the namespace exactly matches the folder structure.

"Route not found" (404 on a route you defined)

1. Why it happens: Typo in the URL, wrong HTTP method, or route not registered/cached from an old state.
2. How to check: Run `php artisan route:list` to see all registered routes.
3. How to fix: Correct the route definition; run `php artisan route:clear` if using cached routes.

"Database connection failed"

1. Why it happens: Wrong credentials in `.env`, database server not running, or wrong host/port.
2. How to check: Run `php artisan tinker` then `DB::connection()->getPdo();`.
3. How to fix: Verify `.env` values match your actual database server; ensure MySQL/Oracle service is running; run `php artisan config:clear` if `.env` changes aren't taking effect.

SQLSTATE errors (e.g. `SQLSTATE[42S02]`)

1. Why it happens: A query references a table/column that doesn't exist.
2. How to check: Compare the error's table/column name against your actual migrations.
3. How to fix: Run pending migrations (`php artisan migrate`), or fix a typo in the model/migration.

Migration errors ("table already exists" / "column already exists")

1. Why it happens: Running a migration that's already been applied, or two migrations conflict.
2. How to check: `php artisan migrate:status` shows which migrations have run.
3. How to fix: Use `php artisan migrate:fresh` in local dev to reset cleanly, or write a proper corrective migration in production (never drop/recreate production data).

Mass assignment error ("Add [field] to fillable property")

1. Why it happens: Trying to mass-assign a field not listed in `$fillable`.
2. How to check: Look at the model's `$fillable` array.
3. How to fix: Add the missing field to `$fillable` (only if it's actually safe to mass-assign).

CORS error (blocked in browser console)

1. Why it happens: React's origin isn't allowed in Laravel's CORS config.
2. How to check: Browser console shows a "blocked by CORS policy" message.
3. How to fix: Add your frontend URL to `allowed_origins` in `config/cors.php`.

401 Unauthorized

1. Why it happens: Missing or invalid authentication token.
2. How to check: Confirm the `Authorization: Bearer <token>` header is actually being sent.
3. How to fix: Ensure React attaches the token correctly (check Axios interceptor); confirm the token hasn't been revoked/logged out.

403 Forbidden

1. Why it happens: User is authenticated but lacks permission for this action.
2. How to check: Review the relevant Policy/Gate/middleware logic.
3. How to fix: Confirm the user's role/ownership matches what the authorization rule expects.

404 Not Found

1. Why it happens: Wrong URL, or `findOrFail()`/route-model-binding couldn't find the record.
2. How to check: Confirm the ID exists in the database and the route matches.
3. How to fix: Correct the URL or handle "not found" gracefully in the frontend.

419 Page Expired

1. Why it happens: A CSRF token mismatch (usually on Blade/session-based forms, not typical token-based API auth).
2. How to check: Confirm `@csrf` is present in the form, or that you're not mixing session auth with a stale session.
3. How to fix: Refresh the page to get a fresh token; for pure APIs, ensure you're using Sanctum tokens, not session-based CSRF-protected routes.

422 Validation Error

1. Why it happens: Submitted data failed one or more validation rules.
2. How to check: Inspect the JSON response's `errors` object for the specific failing fields.
3. How to fix: Correct the request payload to match the validation rules; display field errors in the React form.

500 Server Error

1. Why it happens: An uncaught exception occurred somewhere in the app.
2. How to check: Check `storage/logs/laravel.log`, or temporarily set `APP_DEBUG=true` locally.
3. How to fix: Read the actual error/stack trace and fix the underlying bug; never leave `APP_DEBUG=true` in production.

Storage permission error

1. Why it happens: The web server doesn't have write access to `storage/` or `bootstrap/cache/`.
2. How to check: Look for "Permission denied" in the log when writing files/logs.
3. How to fix: `chmod -R 775 storage bootstrap/cache` and ensure the correct owner (usually the web server user).

Composer errors (dependency conflicts)

1. Why it happens: Incompatible package versions, or outdated `composer.lock`.
2. How to check: Read the specific conflict message Composer prints.
3. How to fix: Run `composer update` cautiously, or adjust version constraints in `composer.json`.

PHP extension errors ("Call to undefined function")

1. Why it happens: A required PHP extension (e.g. `oci8`, `mbstring`, `pdo_mysql`) isn't enabled.
2. How to check: `php -m` lists all enabled extensions.
3. How to fix: Enable the missing extension in `php.ini` and restart the server.

Oracle connection errors

1. Why it happens: Oracle Instant Client not installed/configured, wrong `DB_SERVICE_NAME`, or `oci8` extension not enabled.
2. How to check: `php -m | grep oci8`; test with `php artisan tinker` → `DB::connection('oracle')->getPdo();`.
3. How to fix: Verify Instant Client path is in your system's library path, confirm `.env` Oracle settings match your actual database service name and port.

PART 42 — LARAVEL CHEAT SHEET

Artisan

bash

```bash
php artisan serve
php artisan make:model Name -m
php artisan make:controller NameController --api
php artisan migrate / migrate:rollback / migrate:fresh --seed
php artisan tinker
```

Routes

php

```php
Route::get('/path', [Controller::class, 'method']);
Route::apiResource('products', ProductController::class);
Route::middleware('auth:sanctum')->group(fn() => ... );
```

Controllers

php

```php
public function index() { return response()->json(Model::all()); }
```

Models

```php
class Product extends Model {
    protected $fillable = ['name', 'price'];
}
```

Eloquent

```php
Model::all(); Model::find($id); Model::create([...]);
Model::where('x', $y)->get(); Model::findOrFail($id)->update([...]);
```

Migrations

```php
Schema::create('table', function (Blueprint $table) {
    $table->id(); $table->string('name'); $table->timestamps();
});
```

Validation

```php
$request->validate(['email' => 'required|email|unique:users']);
```

Authentication (Sanctum)

```php
$token = $user->createToken('auth_token')->plainTextToken;
Route::middleware('auth:sanctum')->group(...);
```

Middleware

```php
```

```php
class MyMiddleware {
    public function handle($request, Closure $next) { return $next($request); }
}
```

Relationships

```php
public function posts() { return $this->hasMany(Post::class); }
public function user() { return $this->belongsTo(User::class); }
public function tags() { return $this->belongsToMany(Tag::class); }
```

API Resources

```php
class ProductResource extends JsonResource {
    public function toArray($request) { return ['id' => $this->id]; }
}
```

File Uploads

```php
$path = $request->file('image')->store('folder', 'public');
```

Queues

```php
MyJob::dispatch($data);
php artisan queue:work
```

Jobs

```php
class MyJob implements ShouldQueue { public function handle() { ... } }
```

Events

```php
```

```
MyEvent::dispatch($data);
```

Testing

```
php

$this->postJson('/api/route', [...])->assertStatus(201);
```

PART 43 — LEARNING ROADMAP

Beginner → Intermediate → Advanced

Beginner (know by end of Week 1-2): PHP basics and OOP, Laravel installation, routing, controllers, migrations, basic Eloquent CRUD, Blade basics, `.env` configuration.

Intermediate (know by end of Week 3-4): Relationships (all types), API Resources, Form Requests, Sanctum authentication, authorization (Policies/Gates), file uploads, pagination/search/filter, middleware, basic testing.

Advanced (know by end of the course): Service/Repository architecture, Events/Listeners, Jobs/Queues, Mail/Notifications, advanced security, Oracle integration, deployment, and building/maintaining a full production-grade REST API consumed by React.

30-Day Laravel Learning Plan

Day 1: PHP syntax, variables, data types, strings, arrays. Install PHP + Composer. **Day 2:** PHP conditions, loops, functions, OOP basics (classes, objects, constructors). **Day 3:** PHP inheritance, interfaces, traits, namespaces, exceptions. Install Laravel, run `php artisan serve`. **Day 4:** Laravel project structure, `.env`, MVC, request lifecycle. **Day 5:** Routing — all HTTP verbs, parameters, named routes, groups. **Day 6:** Controllers — creation, resource controllers, dependency injection. **Day 7:** Review week 1; rebuild the Product CRUD API from Part 13 from memory. **Day 8:** Blade basics (even though you're API-focused, understand it). **Day 9:** Database fundamentals + migrations (create, modify, rollback). **Day 10:** Models & Eloquent ORM CRUD. **Day 11:** Eloquent queries — where, orderBy, aggregates. **Day 12:** Relationships — hasOne, hasMany. **Day 13:** Relationships — belongsToMany, eager loading, N+1 problem. **Day 14:** Build a small blog API with Users → Posts → Comments relationships. **Day 15:** Form Request validation. **Day 16:** API Resources and pagination. **Day 17:** Search, filter, sort — build a professional listing endpoint. **Day 18:** Authentication with Sanctum — register/login/logout. **Day 19:** Authorization — Gates, Policies, roles. **Day 20:** File & image uploads. **Day 21:** Connect a real React app to your Laravel API (Axios, CORS, auth flow). **Day 22:** Middleware — build custom role middleware. **Day 23:** Service classes — refactor a controller into a service. **Day 24:** Events, Listeners, Jobs, Queues. **Day 25:** Mail and Notifications. **Day 26:** Error handling and Security best practices. **Day 27:** Seeders, Factories, and Testing (write your first Feature test). **Day 28:** Oracle setup and connection (or deepen MySQL if Oracle access isn't

available yet). **Day 29:** Build the full E-Commerce final project from Part 38, end to end. **Day 30:** Deployment — push to GitHub, deploy to a real server/hosting platform, review the entire cheat sheet and interview questions.

END OF COURSE

You've now covered everything from PHP fundamentals to a full production-style Laravel + React + Oracle/MySQL e-commerce API, including authentication, authorization, file uploads, background jobs, testing, security, deployment, 90 interview questions, and a 30-day study plan.

Suggested next step: Rebuild the Part 38 final project yourself, from scratch, without copying — that's the fastest way to make this knowledge permanent.